

微处理器原理与应用

8086微处理器系统结构

主讲：王鹏伟

章节：1~4章

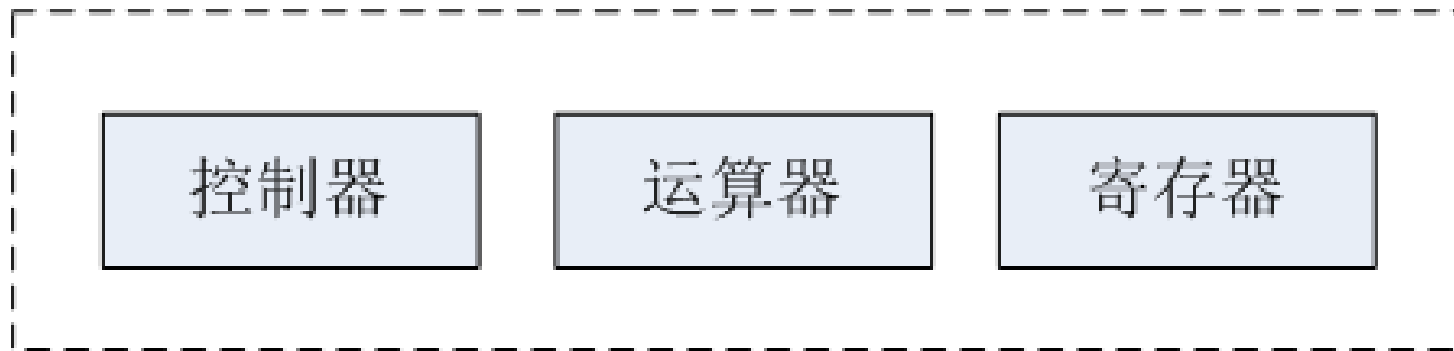
学时：16学时

Email: wangpw@sdu.edu.cn

8086 CPU结构

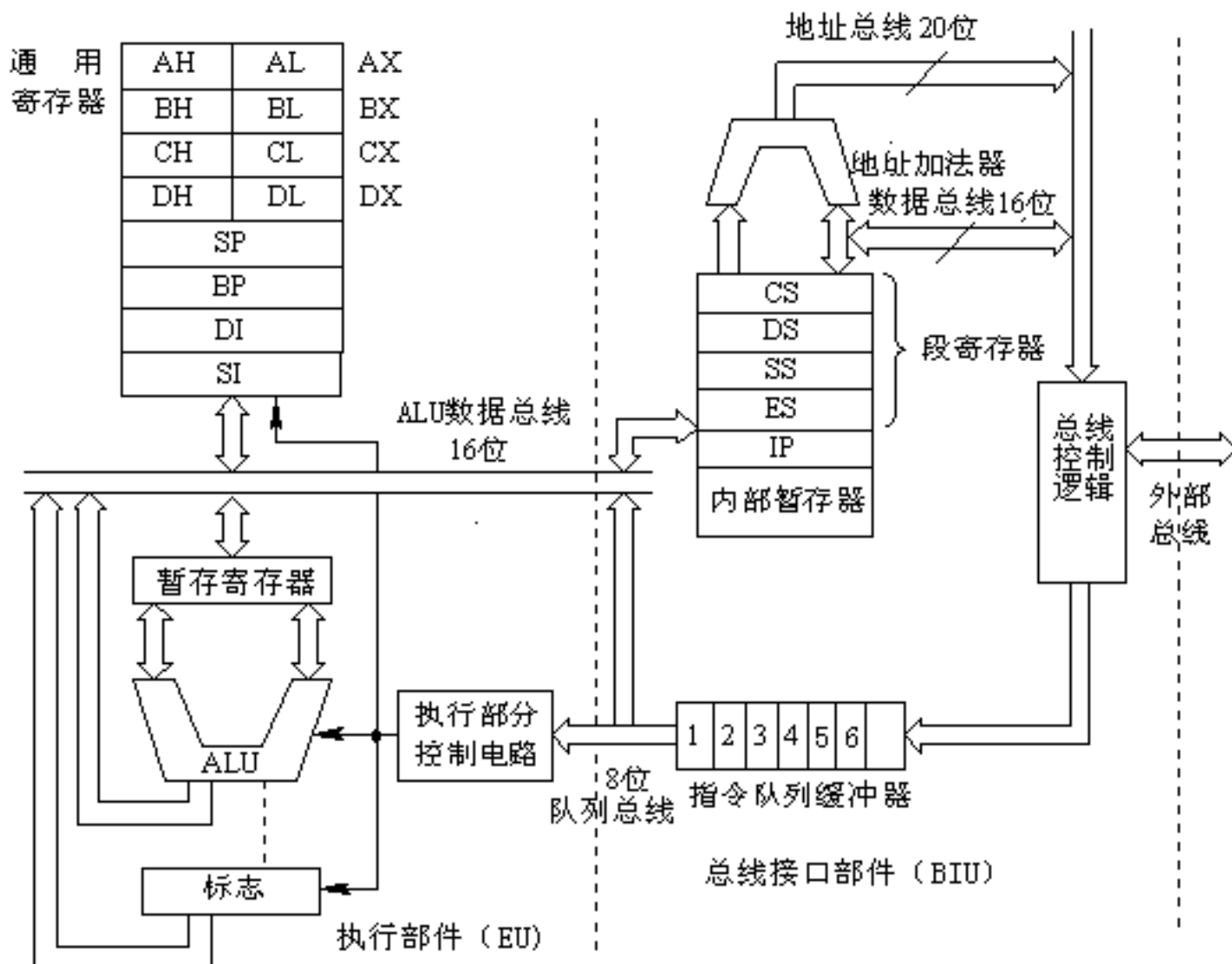
- 8086 CPU是Intel系列的**16位微处理器**，它采用HMOS工艺制造，双列直插，有**40个引脚**。8086 CPU的电源为单一的5V，主时钟频率为5MHz~10MHz。它的**外部数据总线为16位，地址线为20根**。因为可用20位地址，所以可寻址的地址空间达1MB。
- 8086 CPU内部采用了**并行流水线结构**，可以提高CPU的利用率和处理速度。8086 CPU被设计为支持多处理器系统，因此能**方便地与数值协处理器8087或其他协处理器相连**，构成多处理器系统，从而提高系统的数据处理能力

8086处理器的结构



- 控制器用来控制程序和数据的输入/输出，以及各部件之间的协调运行，运算器用来进行算术运算和逻辑运算，并保存中间运算结果，寄存器组则用来临时存放信息，其信息或为要处理的数据，或为内存中获取某数据的地址。

Intel 8086的内部结构框图



总线接口部件BIU

- BIU是CPU与外部存储器及输入输出的接口，负责与存储器和输入输出系统进行数据交换。

BIU由下列各部分组成：

- **4个16位段地址寄存器**，即代码段寄存器**CS**，数据段寄存器**DS**，附加段寄存器**ES**和堆栈段寄存器**SS**，它们分别用于存放当前代码段、数据段、附加段和堆栈段的段基址。
- 段基址表示20位**段起始地址的高16位**，段起始地址的**低4位固定是0**。
- **16位指令指针IP**：**IP**用于存放下一条要执行指令的**有效地址EA(即偏移地址)**，**IP**的内容由**BIU**自动修改，通常是进行加1修改。当执行转移指令、调用指令时，**BIU**装入**IP**中的是转移目的地址。

总线接口部件BIU

- 偏移地址表示离段起始地址之间的距离，用字节数表示。如**偏移地址=0064H**,表示**该地址距离段起始地址有100个字节**，**偏移地址为0就表示该地址为段起始地址**。
- 由**段基址**（段寄存器的内容）和**偏移地址**两部分构成了存储器的逻辑地址，如CS:IP=3000:2000H，CS:IP=0200:1020H等，都是逻辑地址。
- **20位物理地址加法器**：加法器用于将逻辑地址变换成读 / 写存储器所需的20位物理地址，即完成地址加法操作。方法是将某一段寄存器的内容（代表段基址）左移4位（相当乘16）再加上16位偏移地址以形成20位物理地址。
- **6字节的指令队列**：当执行单元EU正在执行指令中，且不需要占用总线时，BIU会自动进行预取下一条或几条指令的操作，并按先后次序存入指令队列中排队，由EU按顺序取来执行。
- **总线控制逻辑**：总线控制逻辑用于产生并发出总线控制信号，以实现了对存储器和输入输出端口的读 / 写控制。

执行部件EU

- **算术逻辑单元ALU**: ALU完成16位或8位的二进制数的算术逻辑运算，绝大部分指令的执行都由ALU完成。在运算时数据先传送至**16位的暂存寄存器**中，经ALU处理后，运算结果可通过内部总线送入通用寄存器或由BIU存入存储器。
- **标志寄存器FR**: 它用来反映CPU最近一次运算结果的状态特征或存放控制标志。FR为16位，其中7位未用。
- **通用寄存器组**: 它包括4个数据寄存器AX、BX、CX、DX，其中**AX又称累加器**，4个专用寄存器，即基址指示器BP、堆栈指示器SP、源变址寄存器SI和目的变址寄存器DI。
- **EU控制器**: 它接收从BIU中指令队列取来的指令，经过指令译码形成各种定时控制信号，**向EU内各功能部件发送相应的控制命令**，以完成每条指令所规定的操作。

BIU和EU的动作管理

- 取指令

BIU从内存取指令，并送到指令队列。取指令时的地址由代码段寄存器CS中的16位段基址的最低位后补4个0，再与指令指针IP中的16位偏移地址在地址加法器中相加得到20位物理地址。然后通过**总线控制逻辑发出存储器读命令**，从而启动存储器，从存储器中取出指令并送入指令队列供EU执行。

BIU和EU的动作管理

- 取操作数或存结果

在EU执行指令过程中需要取操作数或存结果时，先向BIU发出请求，并提供操作数的有效地址，BIU将根据EU的请求和提供的有效地址，**形成20位的物理地址并执行一个总线周期去访问存储器或I/O端口**，从指定存储单元或I/O端口取出操作数送交EU使用或将结果存入指定的存储单元或I/O端口。如果BIU已准备好取指令同时又收到EU的申请，**BIU先完成取指令的操作，然后进行操作数的读写。**

当EU执行**转移、调用和返回指令**时，BIU先自动清除指令队列，再按EU提供的新地址取指令。BIU新取得的第一条指令将直接送到EU中去执行。然后，BIU将随后取得的指令重新填入指令队列。

8086寄存器结构

AH	AL
BH	BL
CH	CL
DH	DL

AX
BX
CX
DX

数据寄存器

SP
BP
SI
DI

栈指针

基数指针

源变址

目的变址

指针寄存器

变址寄存器

IP
FLAG

指令指针

状态标志

控制寄存器

CS
DS
SS
ES

代码段

数据段

栈段

附加段

段寄存器

通用
寄存器

通用寄存器

- 数据寄存器

数据寄存器包括AX、BX、CX、DX等4个16位寄存器，主要用来**保存算术、逻辑运算的操作数、中间结果和地址**。它们既可以作为16位寄存器使用，也可以将每个寄存器高字节和低字节分开作为两个独立的8位寄存器使用。而8位寄存器（AL、BL、CL、DL、AH、BH、CH、DH）**只能用于存放数据**。

- 地址指针和变址寄存器

地址指针和变址寄存器组包括堆栈指针 SP、堆栈基址寄存器 指针BP以及变址寄存器 指针SI和DI等4个16位寄存器。它们主要是用来存放或指示操作数的**偏移地址**。

地址指针和变址寄存器

- 堆栈指针SP中存放的是当前堆栈段中栈顶的偏移地址。堆栈操作指令PUSH和POP就是从SP中得到操作数的段内偏移地址的。
- BP是访问堆栈时的基址寄存器。BP中存放的是堆栈中某一存储单元的偏移地址，SP、BP通常和SS联用。
- SI和DI称为变址寄存器。它们通常与DS联用，为程序访问当前数据段提供操作数的段内偏移地址。SI和DI除作为一般的变址寄存器外，在串操作指令中SI规定用作存放源操作数(即源串)的偏移地址，称为源变址寄存器；DI规定用作存放目的操作数(目的串)的偏移地址，故称之为目的变址寄存器，二者不能混用。由于串操作指令规定源字符串必须位于当前数据段DS中，目的串必须位于附加段ES中，所以SI和DI中的内容分别是当前数据段和当前附加段中某一存储单元的偏移地址。
- 当SI、DI和BP不作指示器和变址寄存器使用时，也可将它们当作一般数据寄存器使用，存放操作数或运算结果。

隐含寻址

- 8个16位通用寄存器在一般情况下都具有通用性。但是，为了缩短指令代码的长度，某些通用寄存器又规定了专门的用途。**隐含寻址就是在指令中隐含地使用了一些通用寄存器，而这些通用寄存器不直接在指令中表现出来。**
- 例如，在字符串处理指令中约定必须用CX作为计数器存放串的长度。这样指令中就不必给出CX寄存器名，缩短了指令长度，简化了指令的书写形式，这种使用方法称为“隐含寻址”。

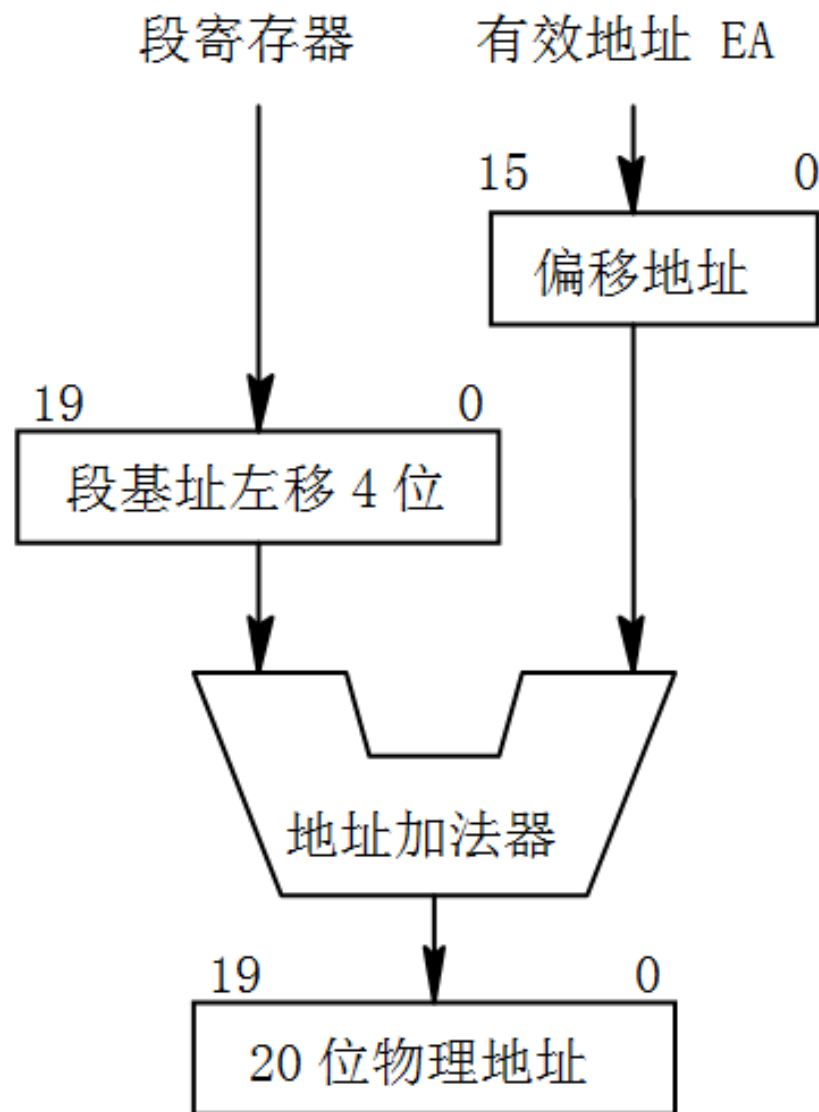
8086 CPU中通用寄存器的特殊用途和隐含性质

AX, AL	在输入输出指令中作数据寄存器	不能隐含
	乘法指令中存放被乘数和乘积, 除法指令中存放被除数和商	隐含
AH	在 LAHF 指令中, 作目标寄存器用	隐含
	在十进制运算指令中作累加器用	隐含
AL	在 XLAT 指令中作累加器用	隐含
BX	在间接寻址中作基址寄存器用	不能隐含
	在 XLAT 中作基址寄存器用	隐含
CX	在串操作指令和 LOOP 指令中作计数器用	隐含
CL	在移位 / 循环移位指令中作移位次数计数器用	不能隐含
DX	在字乘法 / 除法指令中存放乘积高位或被除数高位或余数	隐含
	在间接寻址的输入输出指令中作地址寄存器用	不能隐含
SI	在字符串运算指令中作源变址寄存器用	隐含
	在间接寻址中作变址寄存器用	不能隐含
DI	在字符串运算指令中作目的寄存器用	隐含
	在间接寻址中作变址寄存器用	不能隐含
BP	在间接寻址中作基址指针用	不能隐含
SP	在堆栈操作中作堆栈指针用	隐含

8086物理地址的形成

- 8086 CPU可直接寻址1MB的内存空间。直接寻址时需要20位地址码，而所有的内部寄存器，包括段寄存器，都是16位的，用它们作地址寄存器，只能直接寻址64KB单元。因此，在8086 CPU中采用了存储空间分段技术来解决这一矛盾。将1MB的存储空间分成若干个逻辑段，每段最大长度为64KB。这些逻辑段可在整个1MB存储空间内浮动，但是段的起始地址必须能被16整除。这样对于20位的段起始地址，其低4位为0，可暂时先忽略，而只有高16位是有效数字，可存放于16位的寄存器中。在形成20位物理地址时，段寄存器中的16位数会自动左移4位，然后与16位偏移量相加。

8086物理地址的形成



8086物理地址的形成

- 对于取指操作，是将当前**CS**中的内容左移4位(相当乘16)再加上IP的内容，形成20位指令地址；存取数据操作，是将当前数据段寄存器**DS**中的段基址左移4位，再与16位偏移地址EA相加，形成20位的物理地址；对于压栈和弹栈操作，是将当前堆栈段寄存器**SS**中的段基址左移4位，再与SP相加，形成20位的物理地址；在对目的串操作时，是以当前附加段寄存器**ES**中的段基址左移4位，再与DI相加以形成20位的物理地址。
- 8086还允许部分改变基本段约定，如存取数据的基本段为数据段，但可临时改变为代码段、或附加段、或堆栈段，即数据不仅可在数据段，还可在代码段、附加段和堆栈段中。这种情况称为段超越。

8086的基本段约定和允许的段超越

访问存储器的方式	默认的 段寄存器	可跨越的 段寄存器	偏移地址
取指令	CS	无	IP
堆栈操作	SS	无	SP
一般数据访问	DS	CS ES SS	有效地址EA
BP 作为基址的寻址	SS	CS DS ES	BP
串操作的源操作数	DS	CS ES SS	SI
串操作的目的操作数	ES	无	DI

控制寄存器

指令指针寄存器IP

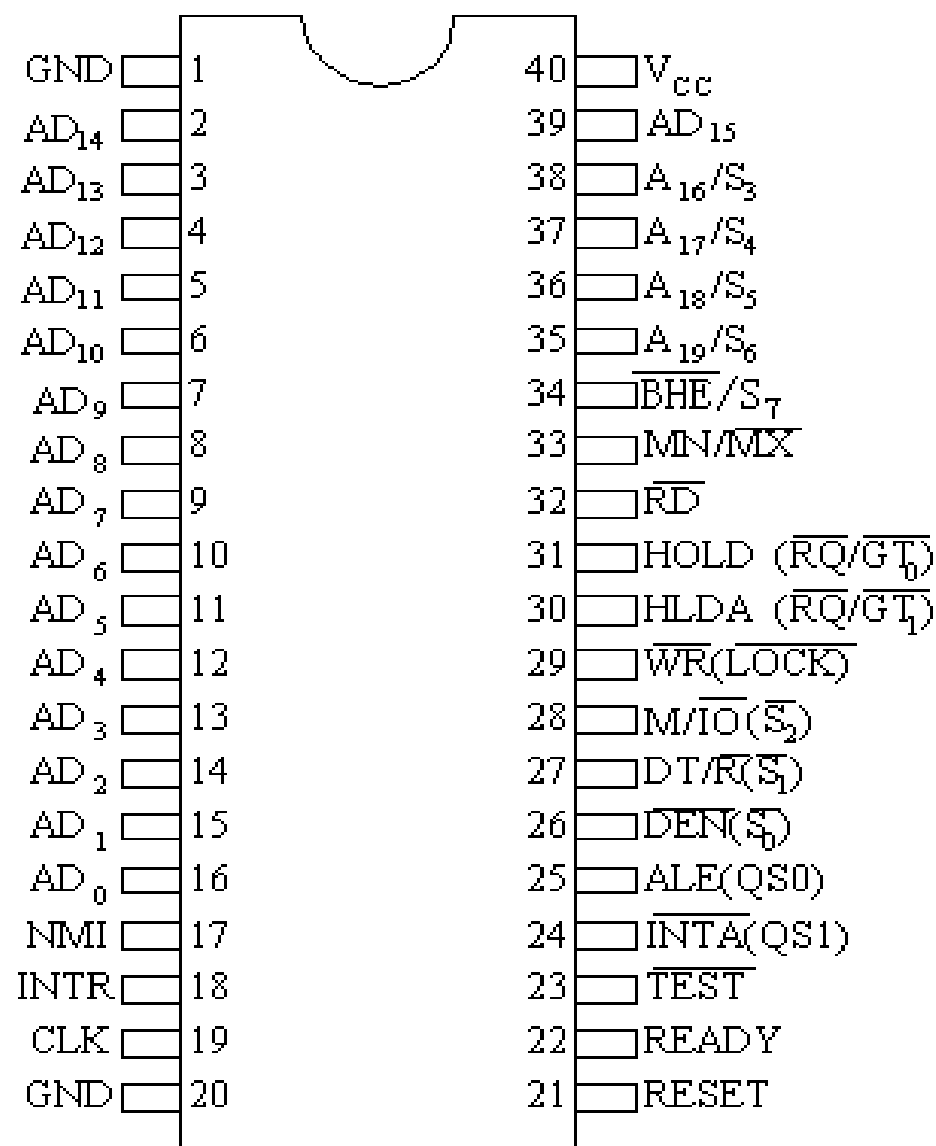
- 在总线接口部件BIU中设置了一个16位的指令指针寄存器IP，其作用是用来存放将要执行的下一条指令在现行代码段中的偏移地址。在程序运行中，IP的内容由BIU自动修改，使IP始终指向下一条将要执行的指令地址。因此，IP实际上起着控制指令流的执行流程。

标志寄存器flags

- 标志寄存器flags也称程序状态字寄存器（简称为PSW），用来存放指令执行结果特征。

15					11	10	9	8	7	6		4		2		0
					OF	DF	IF	TF	SF	ZF		AF		PF		CF

8086 CPU引脚及功能



8086系统时钟与指令周期

- 每条指令的解释执行过程包括**取指令**和**执行指令**两个步骤，CPU取一条指令并执行这条指令，都要完成一系列的操作，这一系列操作所需要的时间通常叫做一个**指令周期**。由于各种指令的操作功能不同，因此各种指令的指令周期是不尽相同的。对应指令执行的三个阶段，指令周期一般分为：**取指周期**、**取操作数周期**和**执行周期**三个部分。

(1) 取指周期

- 取指周期是取出某条指令所需的时间。在取指周期中CPU主要完成两个操作：1) 按程序计数器PC的内容取指令；2) 形成后继指令的地址。

取指周期 = (指令的长度 / 存储字的长度) × 主存的读/写周期

指令周期

(2) 取操作数周期

- 取操作数周期是为执行指令而取操作数所需的时间。取操作数周期的长短与操作数的个数有关、与操作数所处的物理位置有关还与操作数的寻址方式有关。

(3) 执行周期

- 执行周期是完成指令所规定的操作和送结果所需的时间。它与指令规定的操作复杂程序有关，还与目的操作数的物理位置和寻址方式有关。状态信息中的条件码在执行周期中存入程序状态字PSW。若该指令是转移指令，在该周期中还要生成转移地址。

几个周期概念

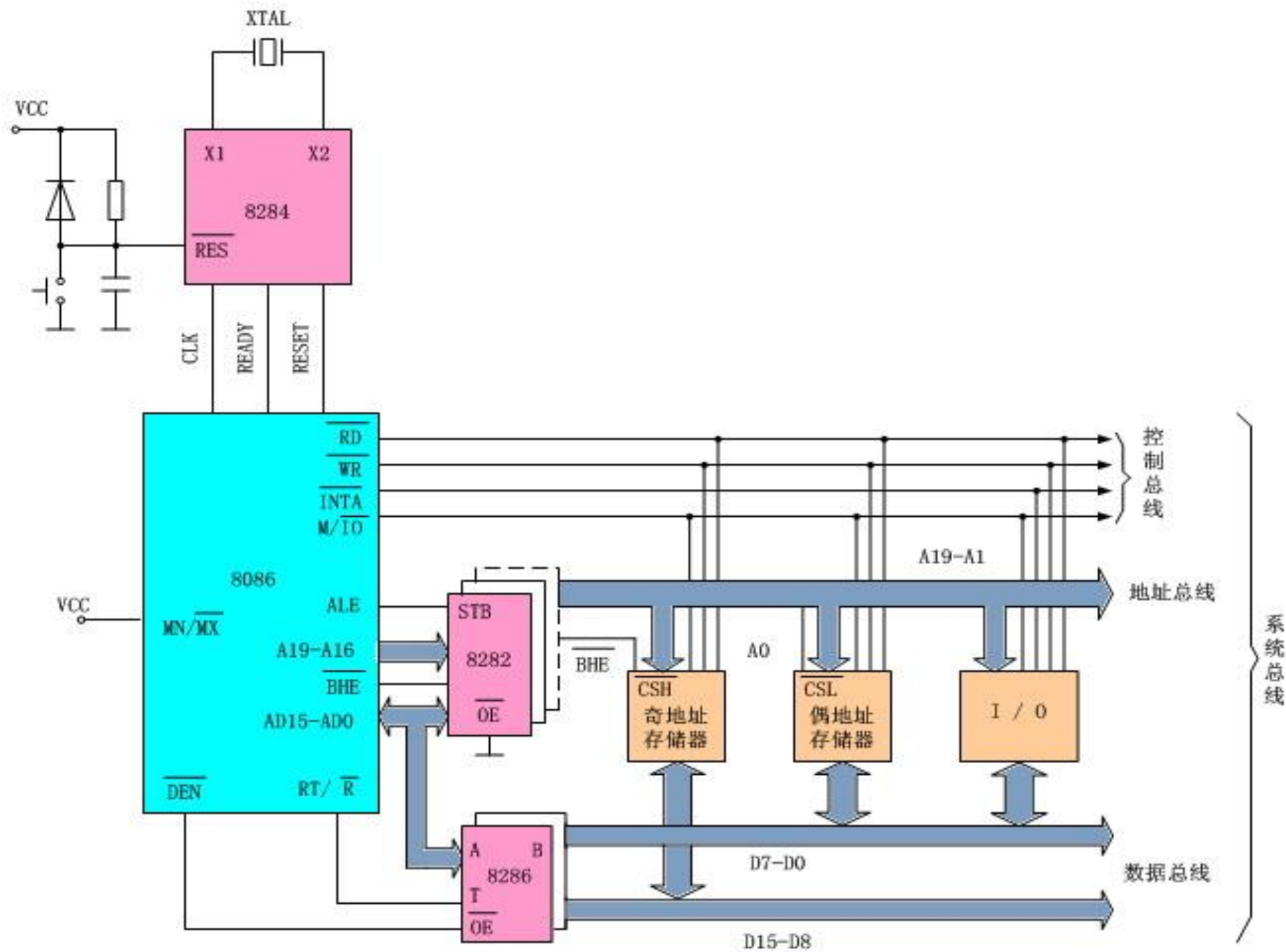
- 指令周期常常用若干个**CPU周期数**来表示，CPU周期也称为**机器周期**。由于CPU内部的操作速度较快，而CPU访问一次内存所花的时间较长，因此通常**用内存中读取一个指令字的最短时间来规定CPU周期**。也就是说，一条指令的取出阶段（通常称为取指）需要一个CPU周期时间。而一个CPU周期时间又包含有若干个**时钟周期（通常称为节拍脉冲或T周期，它是处理操作的最基本单位）**。时钟周期是CPU的时间基准，由计算机的主时钟脉冲决定，执行每条指令和每个总线周期的一系列操作都是在时钟脉冲的同步下进行的。这些时钟周期的总和则规定了一个CPU周期的时间宽度。
- 在执行指令的过程中，CPU要占用系统总线访问内存或外部设备，以便从内存或外设接口中读取指令或指令所需的操作数。**CPU占用一次系统总线，进行信息的输入输出所需要的时间称为总线周期。**

时钟及时钟发生器

- 8086 CPU由外部的一片8284芯片提供主频为5MHZ的时钟信号。8284A是Intel公司专为8086设计的时钟信号发生器，能产生8086所需的系统时钟信号（主频），可采用石英晶体或TTL脉冲发生器作振荡源。8284A除提供恒定时钟信号外，还对外界输入的就绪信号RDY和复位信号RES进行同步。
- 根据不同的振荡源，8284A有两种不同的连接方法：一种是用**脉冲发生器作振荡源**，此时，将脉冲发生器的输出端与8284A的EFI端相连，并且8284A的 $F/\bar{C}$ 端接高电平；另一种方法是用**晶体振荡器作为振荡源**，此时，将晶体振荡器接在8284A的X1和X2两端，并且8284A的 $F/\bar{C}$ 接低电平（地）。

8086 CPU的工作模式

- 最小工作模式用于单机系统，系统中所有总线控制信号全部由8086直接提供，因此系统中的总线控制电路可减到最少；最大工作模式用于多处理机系统，8086 CPU作为主处理器，其它的处理器为协处理器，协助主处理器工作。**在最大工作模式下，系统所需要的控制信号均由总线控制器8288提供。** 8086最小模式下的总线控制信号由CPU直接产生，用于总线控制的信号是HOLD（总线请求信号，输入）、HLDA（总线响应信号，输出）。
- 8086具体工作在最大模式还是最小模式，完全由硬件连接决定。当将CPU的第33号引脚MN/ 接+5V时，8086工作在最小模式，当MN/ 接地时，8086工作在最大模式。



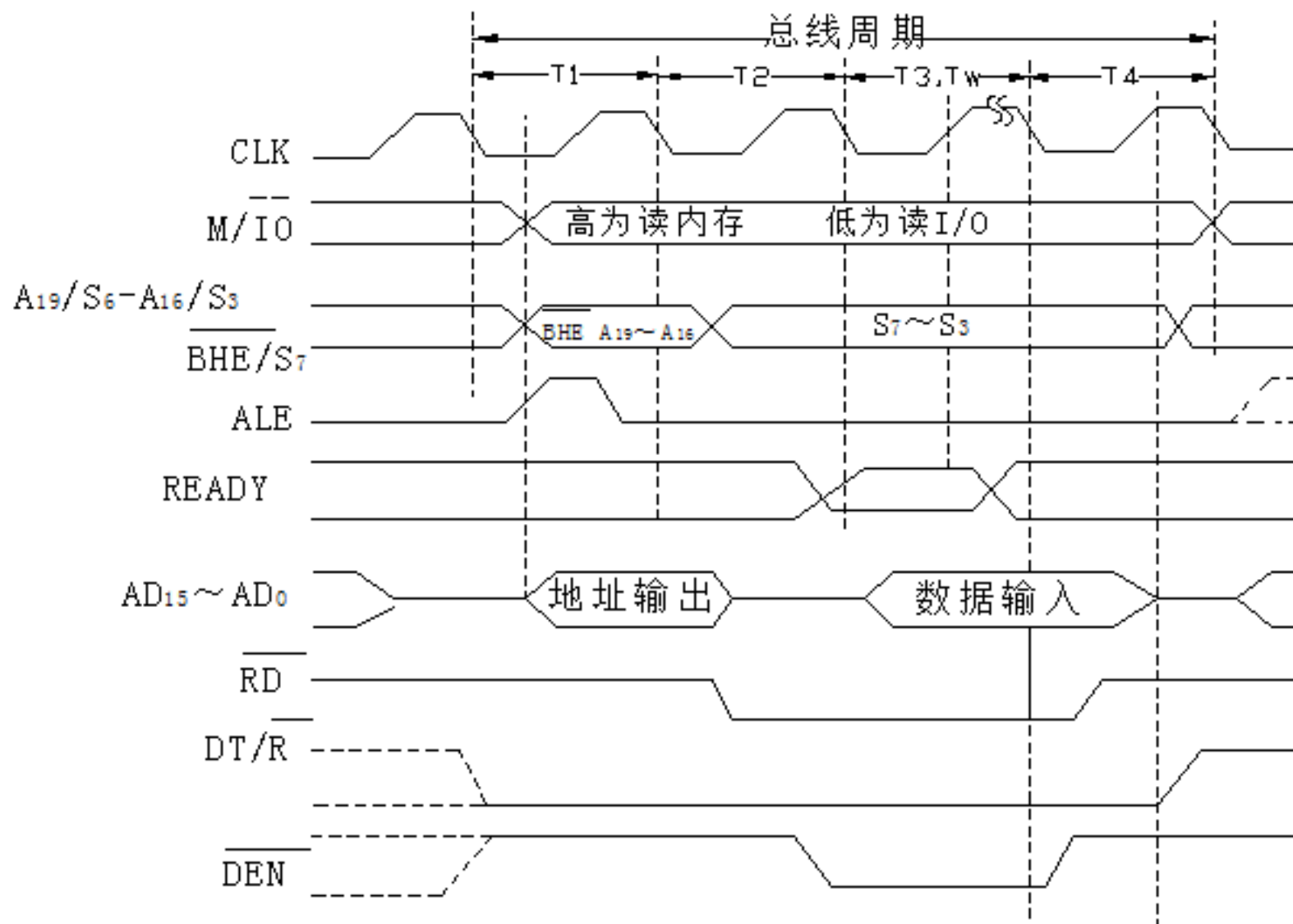
8086 CPU总线操作与时序

- 8086 CPU的操作可分为**内操作**与**外操作**两种，内操作是CPU内部执行指令的过程，外操作是CPU与外部进行信息交换的过程，外部操作主要指的是总线操作。
- 8086主要的总线操作有：系统复位和启动操作、总线读/写操作、总线保持操作或总线请求/允许操作、中断响应操作、暂停操作、空操作。

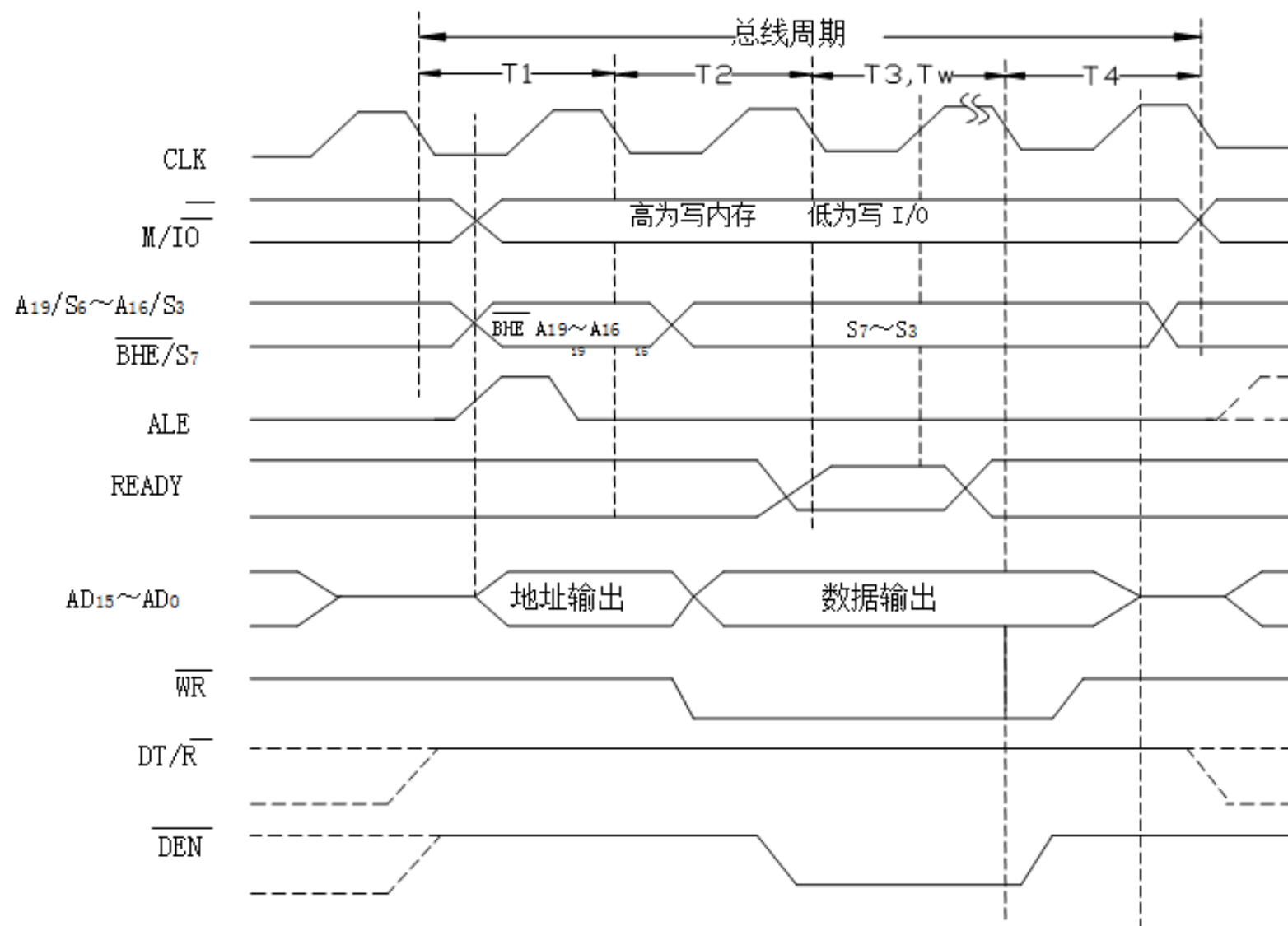
8086基本总线周期

- 8086最基本的总线周期由4个时钟周期组成，分别用T1、T2、T3、T4表示，称为T1、T2、T3、T4状态，T1状态，CPU输出地址信号，T2、T3、T4状态传送数据，如果在T2、T3、T4状态无法完成数据传送，就在T3与T4状态之间插入Tw。
- Tw状态也叫等待状态或等待周期，是CPU在读写存储器或与外设交换信息时，为了与存储器或外设的速度匹配，在T3状态之后插入的1个或多个等待周期。**Tw状态的插入是通过系统中的等待信号产生电路产生一个WAIT信号，经时钟发生器8284同步后传递给CPU的READY线来实现的**，此时，总线上的状态一直不变，用于数据传送，当CPU接到有效的READY信号后，数据传送结束，进入T4状态。T4状态后，就要进入下一个总线周期的T1状态，否则，就进入空闲状态，称为Ti状态。

8086读总线周期



8086写总线周期



8086总线请求与响应

- 8086最小模式下的总线控制信号由CPU直接产生，用于总线控制的信号是HOLD（总线请求信号，输入）、HLDA（总线响应信号，输出）。当系统中其他部件，如DMA控制器，需要占用总线时，向CPU发出总线请求信号HOLD。CPU收到有效的HOLD信号后，如果允许让出总线，就在当前总线周期完成时，发出HLDA信号，同时使地址/数据总线和控制总线处于高阻态，表示让出总线，在下一个时钟周期，总线请求部件收到HLDA信号，获得总线控制权。在这期间，HOLD和HLDA都保持高电平，直到总线请求部件完成对总线的占用后，使HOLD变为低电平，撤消总线请求，CPU收到后，HLDA信号才变为低电平，CPU恢复对总线的控制。

存储器

- 存储器是计算机实现记忆功能的核心部件，它用于存放待加工的原始数据和中间计算结果以及系统或用户程序等。

计算机的存储器分为两类：

1、内部存储器

用于存储当前与CPU频繁交换的信息，其工作速度快，但容量较小；

2、外部存储器

用于存储CPU暂不处理的信息，其容量很大，故称为海量存储器。外存要配置专门的接口和驱动设备才能实现访问，存取速度也较内存慢得多。当其信息需要处理时，要先调入内存，再由CPU处理。

存储器

- 内部存储器，也称为**内存**，是主存储器，位于计算机主机的内部，用来存放系统软件和当前正在使用的或者经常使用的程序和数据，内存的容量大小受到地址总线位数的限制，例如8086 CPU有20条地址总线，可寻址空间为1MB。



8086存储器组织

存储器地址的分段



存储器地址的分段

- 任何一个存储单元的实际地址，都是由段地址及段内偏移地址两部分组成的。存储单元可以在一个段中定义，也可以定义在两个段的逻辑段中，关键是看段的首地址是如何指定的。IBM PC机对段的首地址有限制，规定必须从每小段（paragraph）的首地址开始，每16个字节为一小段，所以段起始地址必须能被16整除。

物理地址的形成方法

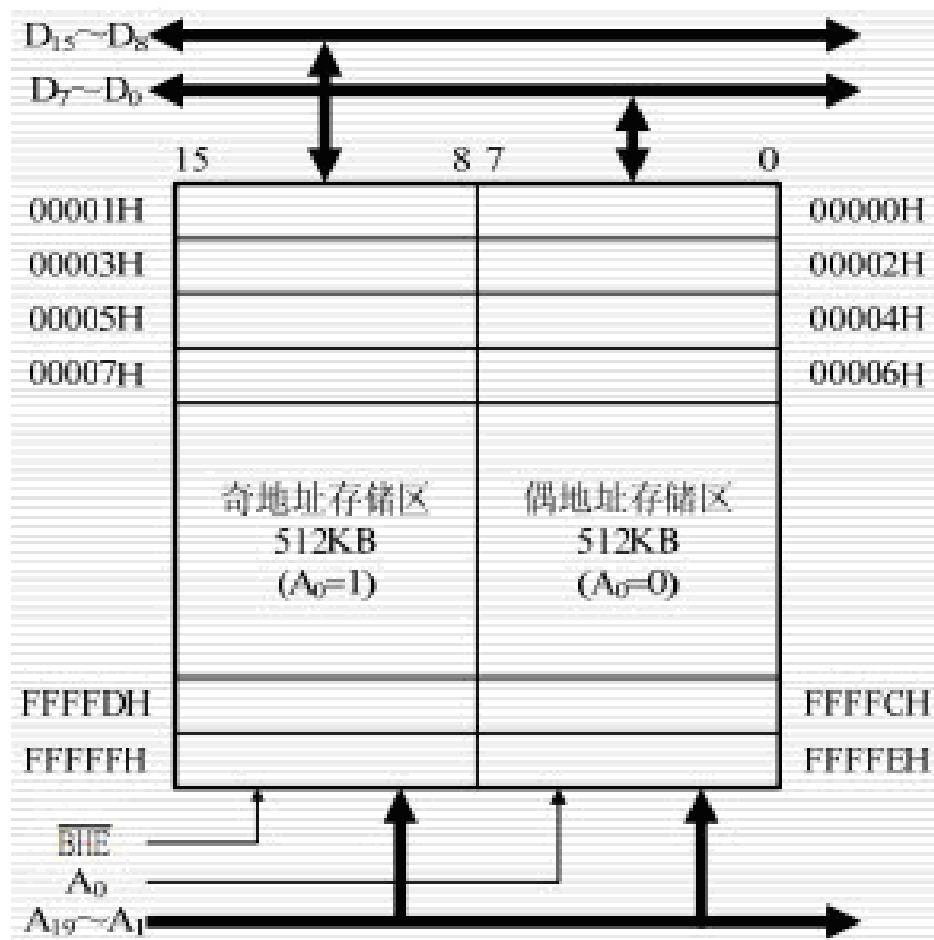
- 8086系统将段地址放在段寄存器中，称“段基址”。有4个段寄存器，分别为代码段寄存器CS，数据段寄存器DS，附加段寄存器ES和堆栈段寄存器SS。
- 段内“偏移地址”指出了从段地址开始的相对偏移位置。它可以放在指令指针寄存器IP中，或者放在16位通用寄存器中，如何从16位段地址和16位偏移地址得到20位地址，需要先说明逻辑地址和物理地址两个概念。
- **逻辑地址：**存储器的任一个逻辑地址是由段基址和偏移地址组成的，都是无符号的16位二进制数，程序设计时采用逻辑地址。
- **物理地址：**存储器的绝对地址，从00000~FFFFFFH，是CPU访问存储器的实际寻址地址，是由逻辑地址变换而来的。
物理地址 = 段基址 16 + 偏移地址

8086存储器的分体结构

- 在8086系统中，CPU有**16根数据线**，而存储器一个基本单元（字节）有**8根数据线**，因此通常将存储系统分为偶体存储器和奇体存储器与8086相连的方法。
- 偶体存储器：一个存储体包含全部偶数地址的单元，奇体存储器：一个存储体包含全部奇数地址的单元。根据8086的规定，**偶地址单元的数据在数据线低8位上传输，奇地址单元的数据在高8位数据线上传输。**
- 偶体存储器的**8根数据线**连接到**数据总线的低8位**上，地址线最低位 **A0** 用作偶体存储器的选择信号，奇体存储器的**8根数据线**连接到**数据总线的高8位**上，8086引脚用 **$\overline{\text{BHE}}$** 低电平作为奇体存储器的选择信号。

8086存储器的分体结构

- 8086系统中，1MB的存储空间分成两个存储器：偶地址存储器和奇地址存储器，分别为512KB。



8086存储器的分体结构

- 存储器中存放的信息称为存储单元的内容，例如，存储单元00100H中的内容为34H，表示为 $(00100H) = 34H$ 。一个字在存储器中按相邻两个字节存放，且存入时以低位字节在低地址，高位字节在高地址的次序存放，字单元的地址以低位地址表示。一个字可以从偶地址开始存放，也可以从奇地址开始存放，**但8086 CPU访问存储器时，都是以字为单位进行的，并从偶地址开始。当CPU读/写一个字时，如果字单元地址从偶地址开始，那么只需要访问一次存储器，低位字节在偶地址单元，高位字节在奇地址单元。如果字单元地址从奇地址开始，那么CPU需要两次访问存储器，第一次取奇地址上数据（忽略偶地址的8位数据），第一次取偶地址上数据（忽略奇地址的8位数据），因此为了加快程序运行速度，编程时注意从存储器偶地址开始存放字数据，这种存放方式也称作“对准存放”。**

栈的概念

- 栈主要用于暂存数据和过程调用或处理中断时暂存断点信息。段基址由SS给出，栈指针由SP给出，指向栈的栈顶。

(1) 栈的概念

- 栈是在存储器中开辟的一片数据存储区，用来存放需要暂时保存的数据。这片存储区的一端固定，另一端活动，且只允许数据从活动端进出。采用“先进后出”的规则。栈段是由段定义的一个段，它可以在存储器1MB空间内任意浮动，但**容量小于等于64KB**。

栈的组织

栈段中存储单元的地址由寄存器SS和SP确定。栈寄存器SS存放的是栈的基地址，表明栈所在的逻辑段；栈指针寄存器SP存放的是栈顶的地址，即始终指向最后推入堆栈的数据所作的单元。存储单元的地址由 $(SS) \times 16 + (SP)$ 形成。**根据栈构成方式不同，栈指针SP指向的可以是当前栈顶单元，也可以是栈顶上的一个“空”单元，一般采用SP指向当前栈顶单元，**栈的伸展方向既可以从大地址向小地址，也可以从小地址向大地址。**8086的堆栈的伸展方向是从大地址向小地址。**

栈具体操作

栈操作以字为单位进行，而栈中的数据必须按规则字存放。
低字节在偶地址单元，高字节在奇地址单元。把数据推入栈为“压入”；从栈取数据称为“弹出”。“压入”数据的时候，先修改SP的值，即将 $(SP)-2$ 送SP，然后再与SS形成存储器的物理地址，将数据存入。“弹出”数据的时候，先从当前SS和SP形成的物理地址上取出数据，然后修改SP，即将 $(SP)+2$ 送SP。栈操作是按“后进先出”的规则进行的。8086 CPU设有专用的指令执行“压入”和“弹出”的操作，在这些指令中，SP的修改是自动进行的。

堆栈的注意点

1、先进后出 2、PUSH POP成对出现

例如：在某段子程序中需要保护BX、CX、DI的内容

```
PUSH    BX
```

```
PUSH    CX
```

```
PUSH    DI
```

```
.....
```

```
POP     DI
```

```
POP     CX
```

```
POP     BX
```

存储器组织与I/O结构

1、芯片地址线的连接

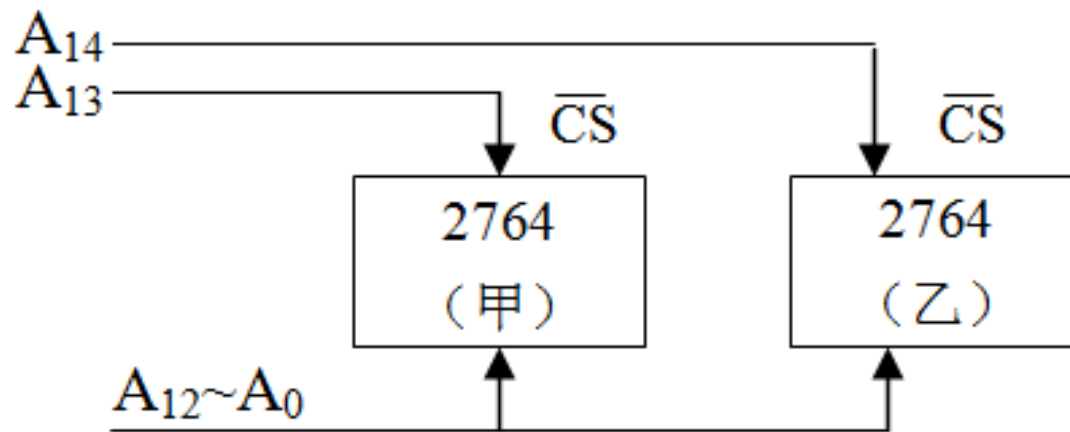
- 存储芯片的地址线通常与系统的低位地址总线相连。寻址时，这部分地址的译码是在存储芯片内部完成的，称为**片内译码**。设某存储器有 N 条地址线，该芯片被选中时。其地址线得到 N 位地址信号，芯片内部进行 $N \rightarrow 2^N$ 的译码，译码后的地址范围是：000...000（ N 位全为0）到111...111（ N 位全为1）。

2、芯片片选端的处理

- 由一个存储芯片或芯片组构成存储器，其容量是有限的，使用时不一定能够满足要求。因此，常要在地址方向上加以扩充，以满足**对多个存储器芯片或芯片组进行寻址**。这一寻址过程中，主要通过将系统高位地址线经外部地址译码器产生的输出信号与存储器的片选端相连接的方法来实现。通过地址译码实现片选的方法有3种：**线性选择法、全译码选择法和部分译码法**。

线性选择法

- 线性选择法是指利用地址总线的高位地址线中的某一位直接作为存储器芯片的片选信号 ($\overline{CS}$)，用地址线的低位实现片内寻址。线选法的优点是结构简单，缺点是地址空间浪费大。由于部分地址线未参与译码，必然会出现地址重叠。此外，当通过线选的芯片增多时，还有可能出现可用地址空间不连续的情况。



全译码选择法和部分译码法

2) 全译码选择法

- 全译码选择法是指将地址总线中**除片内地址以外的全部高位地址接到译码器的输入端参与译码**。采用全译码法，每个存储单元的地址都是唯一的，不存在地址重叠，但译码电路较复杂，连线也较多。

3) 部分译码法

- 部分译码法是将**高位地址线中某几位（而不是全部高位）地址经译码器译码，作为存储器的片选信号**。对被选中的芯片而言，未参与译码的高位地址线可以为0，也可为1，即每个存储单元将对应多个地址。使用时一般将未用地址设为0。采用部分译码法，可简化译码电路，但由于地址重叠，会造成系统地址空间资源的部分浪费。

连接时应注意的问题

1、CPU总线的负载能力

在小型系统中，CPU可直接与存储器芯片连接，但与大量的存储器连接时就应考虑总线的驱动问题。

2、CPU时序与存储器芯片存取速度的配合问题

在微机工作过程中，CPU对存储器的读/写操作是最频繁的操作。因此，考虑存储器与CPU连接时，必须考虑存储器芯片的工作速度是否能与CPU的读/写时序相匹配问题，应从存储器芯片工作时序和CPU时序两个方面来考虑。

3、存储器的地址分配和片选问题

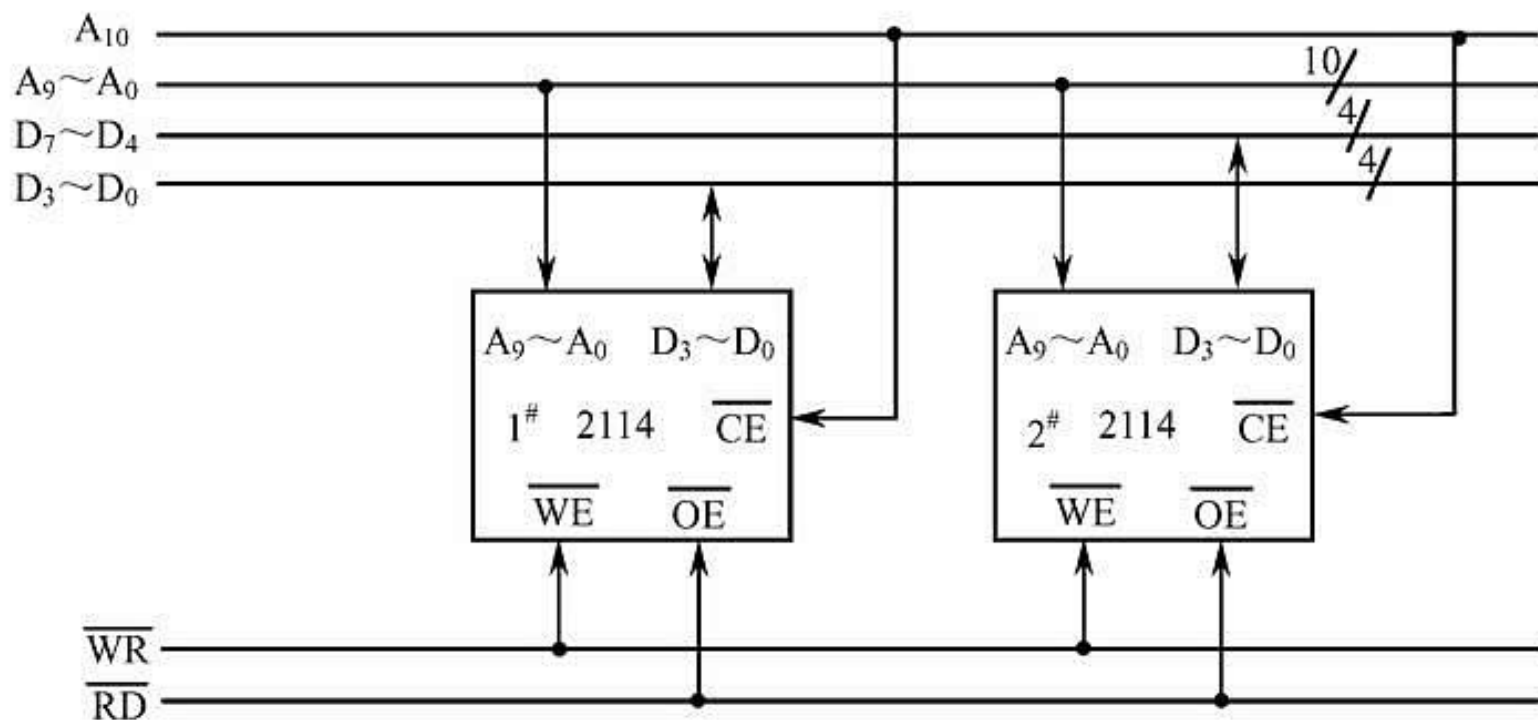
首先确定整机存储容量，再确定选用存储芯片的类型和数量，之后划分RAM、ROM区，画出地址分配图。

4、控制信号的连接

位扩展

位扩展是指增加存储字长。位扩展可以利用芯片地址并联的方式实现，即将**各芯片的数据线分别接到数据总线的各位**，而各芯片的地址线、读/写信号线和片选信号线对应地并联在一起。

- 例如：两片 $1\text{K} \times 4\text{bit}$ SRAM芯片2114，组成 $1\text{K} \times 8\text{bit}$ 存储器



位扩展

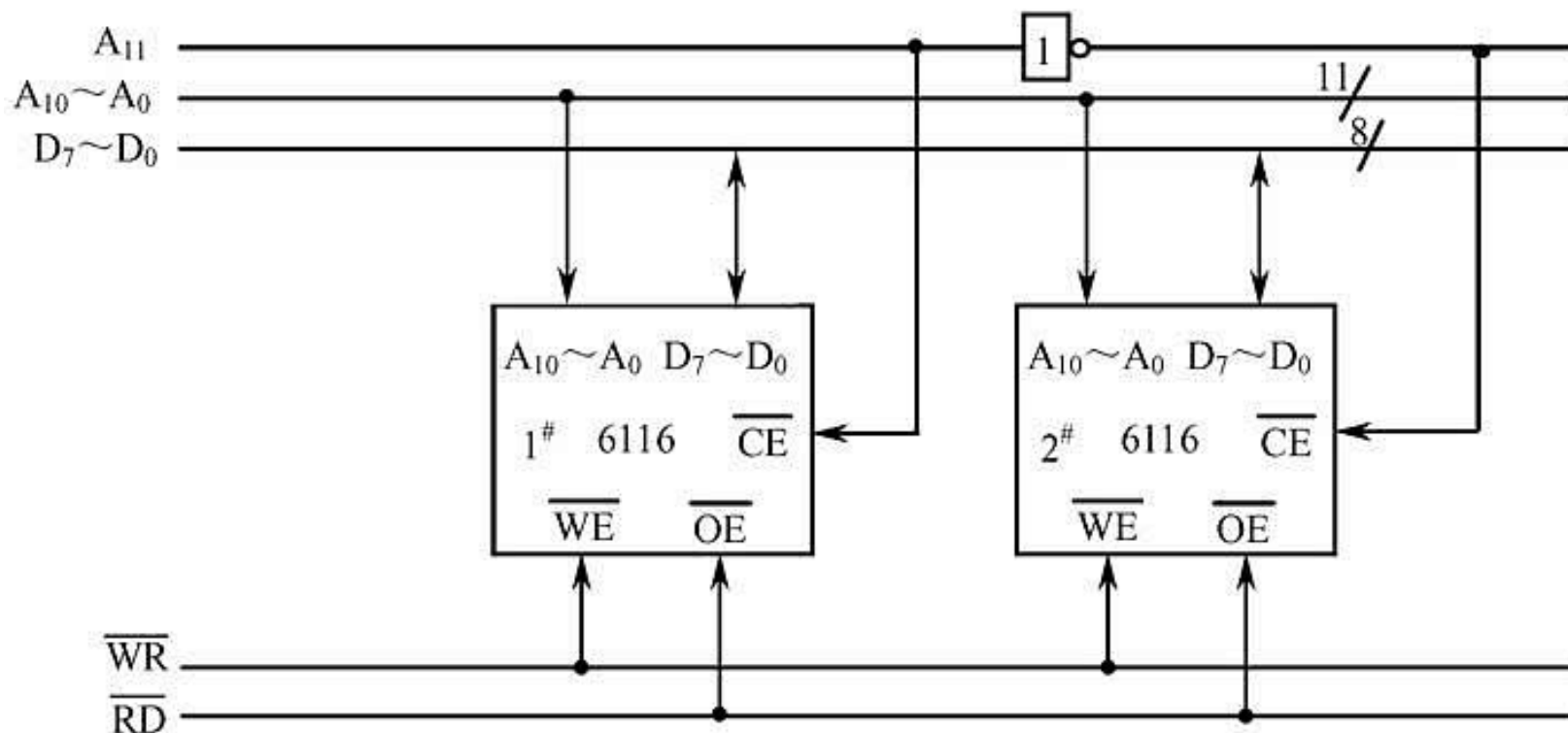
存储器位扩展设计如下图所示。图中两片2114的地址线和各控制线分别并联在一起，而其中**1# 芯片的数据线接数据总线的低4位，2# 芯片的数据线接数据总线的高4位**。硬件连接之后便可确定存储单元的地址，即A9~A0的编码状态000H~3FFH就是1KB存储单元的地址。地址分配情况见下表所示。

A ₁₀	A ₉ ~A ₀	地址范围
0	0000000000	000H
⋮	⋮	⋮
0	1111111111	3FFH

字扩展

字扩展是指增加存储器字的数量，字扩展可利用**芯片地址串联的方式实现**。

例如：用两片 $2K \times 8b$ 的RAM芯片6116组成 $4K \times 8b$ 的存储器



字扩展

字扩展设计如下图所示。图中两片6116的片内信号线A10~A0、D7~D0、 $\overline{OE}$ 、 $\overline{WE}$ 分别与系统的地址线A10~A0、数据线D7~D0和读/写控制线、连接。

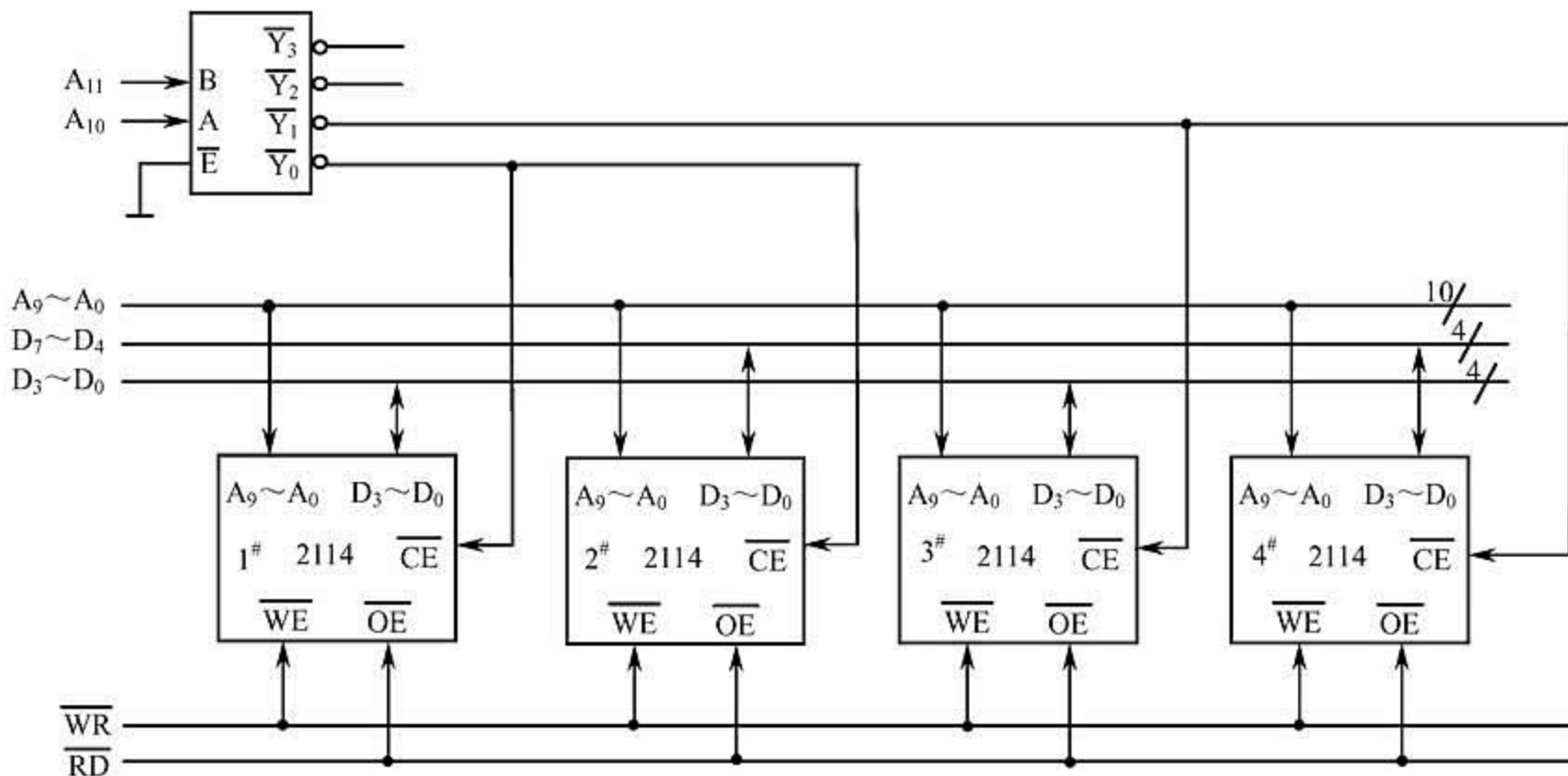
1#芯片的片选信号线与A11连接，2#芯片的片选信号线与A11反相之后连接。当A11为低电平时，选择1#芯片读/写；当A11为高电平时，则选择2#芯片读/写。由下图可见，1#芯片的地址范围是**000H~7FFH**，2#芯片的地址范围是**800H~FFFH**。地址分配情况见下表所示。

芯片号	A ₁₁	A ₁₀ ~A ₀	地址范围
1#	0	00000000000	000H
	⋮	⋮	⋮
	0	11111111111	7FFH
2#	1	00000000000	800H
	⋮	⋮	⋮
	1	11111111111	FFFH

字和位扩展

字和位扩展是字扩展和位扩展的组合。

例如：四片 $1\text{K} \times 4\text{bit}$ RAM 芯片 2114，组成 $2\text{K} \times 8\text{bit}$ 存储器。



字和位扩展

字和位扩展设计如图所示。图中，1#和2#芯片为一组，3#和4#芯片为一组，片内A9~A0、 $\overline{OE}$ 和 $\overline{WE}$ 与系统地址线A9~A0、读/写控制线 $\overline{RD}$ 和 $\overline{WR}$ 对应连接。**1#和3#芯片的数据线D3~D0作为低4位，与系统数据线D3~D0连接；2#和4#芯片的数据线作为高4位，与系统数据线D7~D4连接；1#和2#芯片的 $\overline{CE}$ 连在一起，与2-4线译码器输出端 $\overline{Y}_0$ 连接；3#和4#芯片 $\overline{CE}$ 连在一起，与2-4线译码器输出端 $\overline{Y}_1$ 连接；**系统的高位地址线A11和A10作为2-4线译码器的输入。当A11A10=00时，选择1#和2#的芯片读/写；当A11A10=01时，选择3#和4#芯片读/写。地址分配情况见下表。

存储器字和位扩展地址分配情况表

芯片号	$A_{11}A_{10}$	$A_9 \sim A_0$	地址范围
1#和 2#	00	0000000000	000H
	$\vdots$	$\vdots$	$\vdots$
	00	1111111111	3FFH
3#和 4#	01	0000000000	400H
	$\vdots$	$\vdots$	$\vdots$
	01	1111111111	7FFH

存储器扩展设计步骤

- ① 根据系统实际装机存储容量，确定存储器在整个存储空间中的位置。
- ② 选择合适存储器芯片，列出**地址分配表**。
- ③ 按照地址分配表选用译码器件，画出相应的地址位图，依次**确定片选和片内单元的地址线**，进而**画出片选译码电路**。
- ④ 画出存储器与CPU系统总线的**连接图**。

存储体与总线的连接

为某8位机（地址总线为16位）设计一个32KB容量的存储器。要求采用2732芯片构成**8KB EPROM区**，地址从**0000H**开始；采用6264芯片构成**24KB RAM区**，地址从**2000H**开始。片选信号采用**全译码法**。

- * 每片2732提供 $2^{12} \times 8$ 位(**4K × 8bit**)存储容量
- * 每片6264提供 $2^{13} \times 8$ 位(**8K × 8bit**)存储容量

存储体与总线的连接

- 第一步，确定**实现24KB RAM存储体**所需要的RAM芯片的数量。

因为每片6264提供 $2^{13} \times 8$ 位的存储容量，所以实现24KB存储容量所需要的RAM芯片数量是 $\frac{24 \times 8}{8 \times 8}$

所以RAM数量 = 3（片）

- 第二步，确定**实现8KB ROM存储体**所需要的 $\frac{8 \times 8}{4 \times 8}$ EPROM芯片数量。

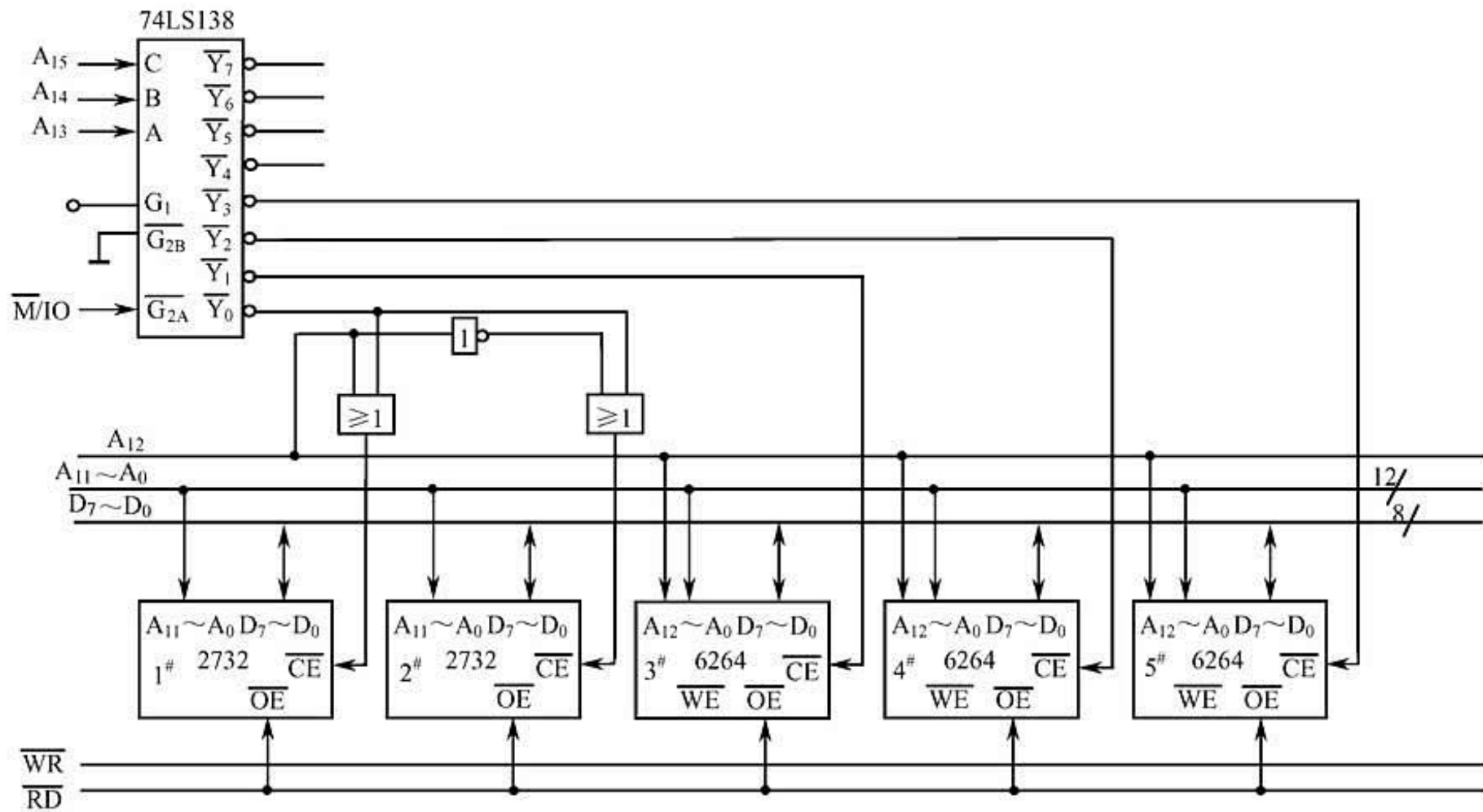
由于每片2732提供 $2^{12} \times 8$ 位的存储容量，所以实现8KB存储容量所需要的EPROM芯片数量是 EPROM数量 = 2（片）

- 第三步，存储器芯片片选择信号的产生及电路设计采用74LS138译码器全译码的方法产生片选信号。

存储器扩展电路

两片2732的片内地址A11~A0与系统地址线A11~A0连接，译码器输出端 $\overline{Y}_0$ 和A12经“或门”输出与1#2732的 $\overline{CE}$ 连接。A12反相后和译码器输出端 $\overline{Y}_0$ 经过“或门”输出与2#2732的 $\overline{CE}$ 连接。三片6264的片内地址A12~A0与系统地址线A12~A0连接，它们的片选 $\overline{CE}$ 分别连接译码器的输出端 $\overline{Y}_1, \overline{Y}_2, \overline{Y}_3$ ，系统地址线A15~A13连接译码器74LS138的输入端A, B, C。

存储器扩展电路

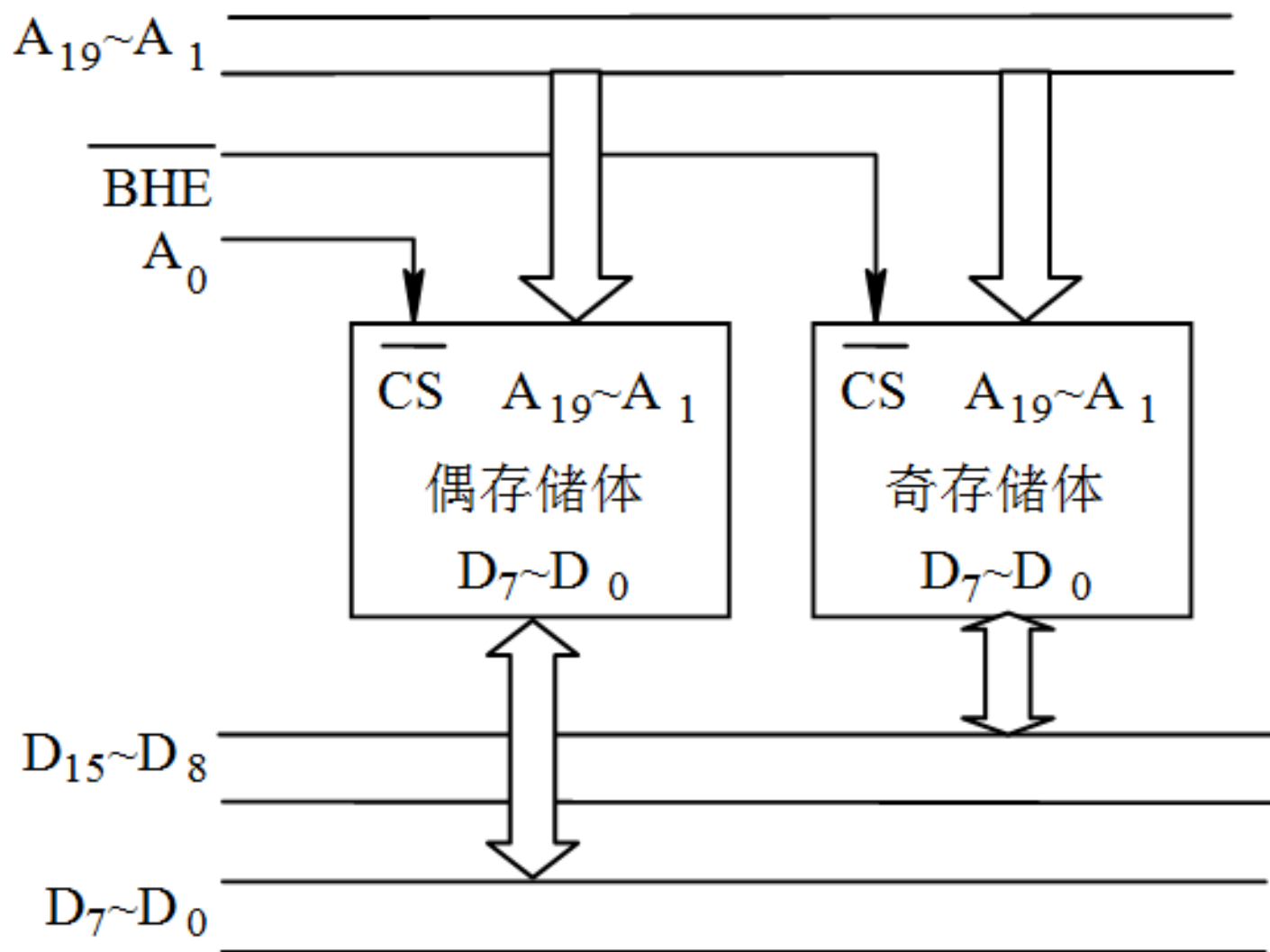


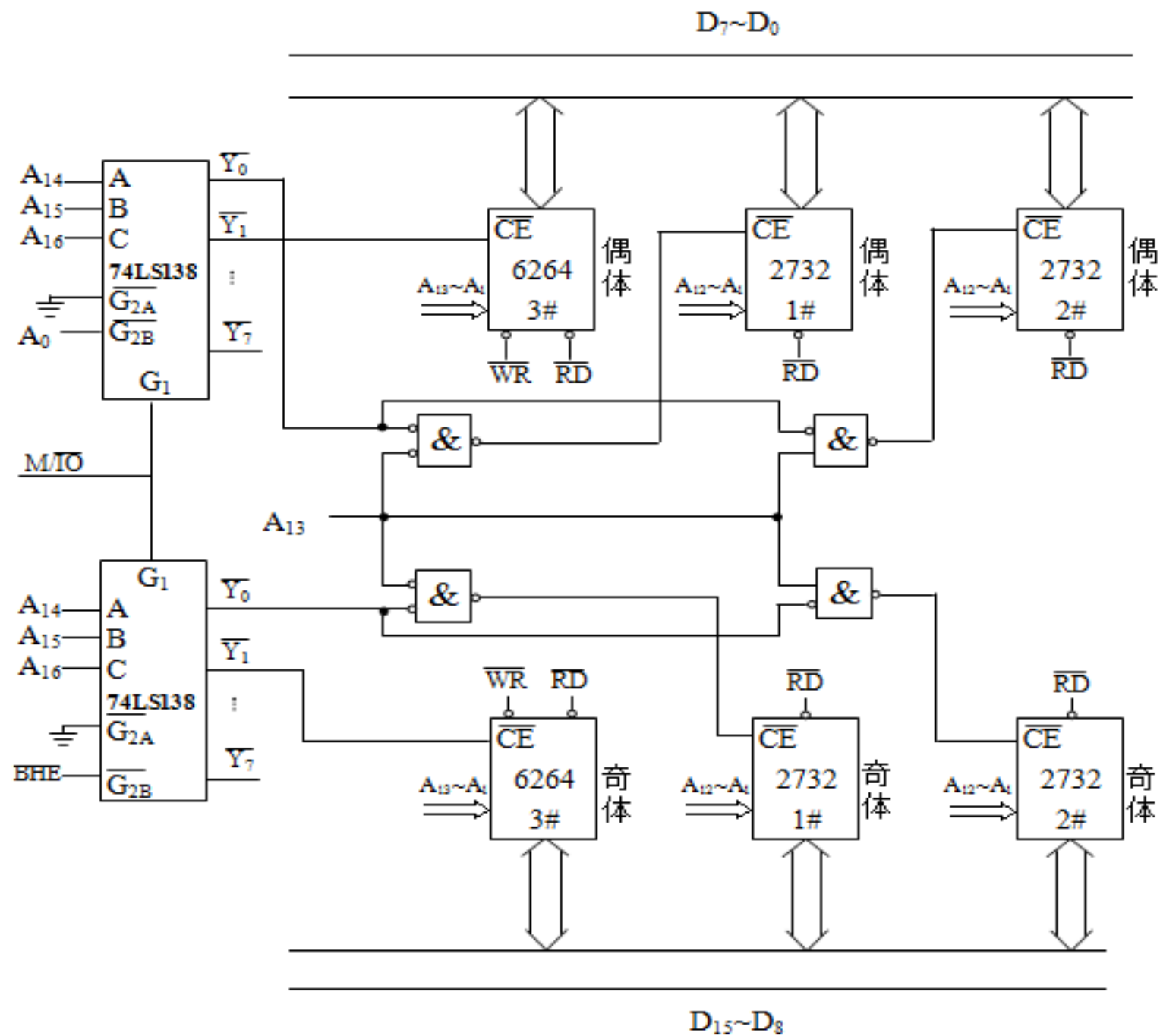
存储器扩展电路

A12~A0作为片内地址线，A15~A13作为3-8译码器74LS138的输入，产生的译码输出000~011作为芯片的片选信号。

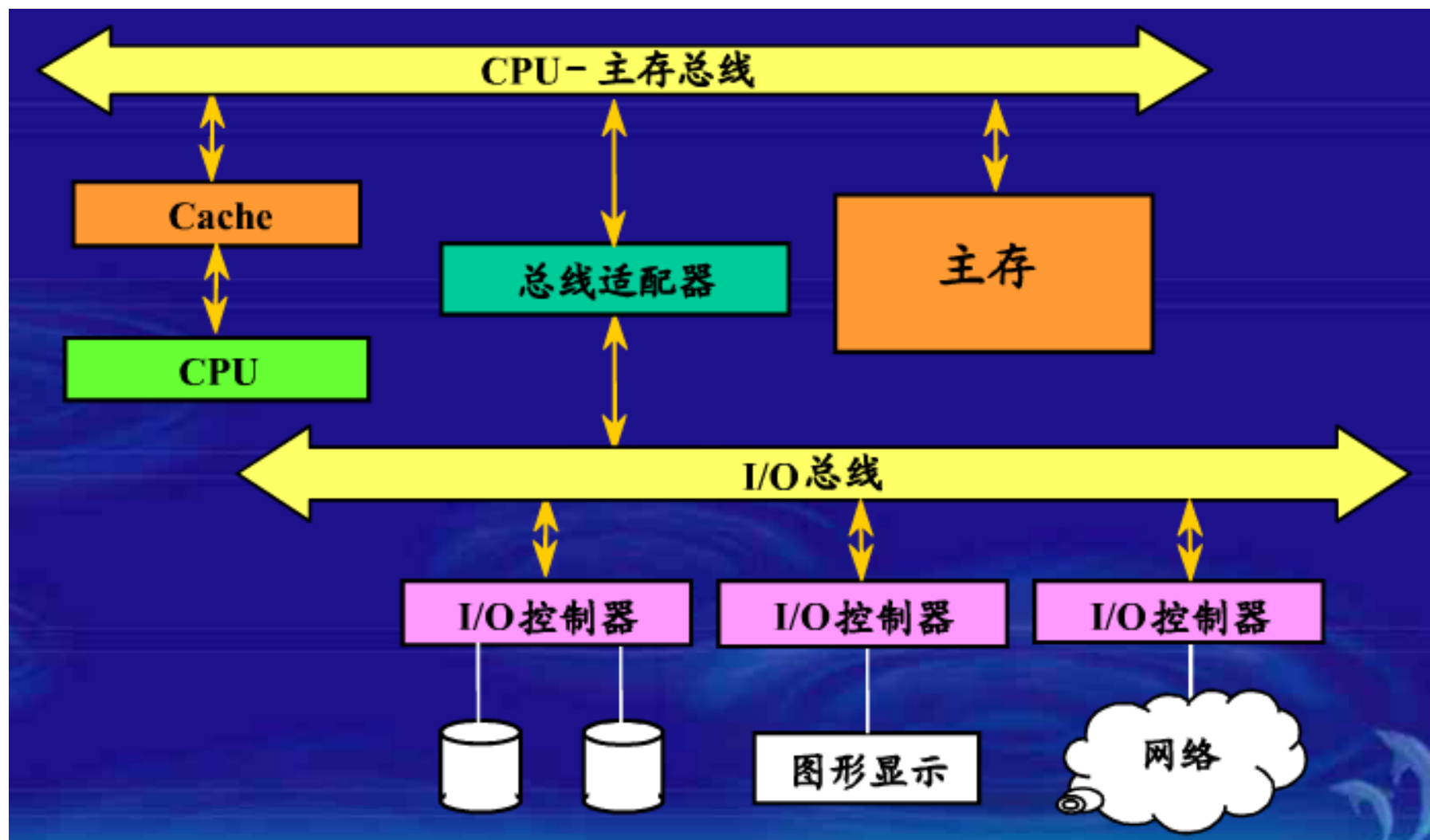
	← 片选译码 →	← 片内译码 →		
芯 片	A ₁₅ ~A ₁₃	A ₁₂	A ₁₁ ~A ₀	地址范围
1 [#]	000	0	00 ... 0	0000H ~ 0FFFH
2 [#]	000	1	11 ... 1	1000H ~ 1FFFH
3 [#]	001	00 ... 0 ... 11 ... 1		2000H ~ 3FFFH
4 [#]	010			4000H ~ 5FFFH
5 [#]	011			6000H ~ 7FFFH

存储体与总线的连接





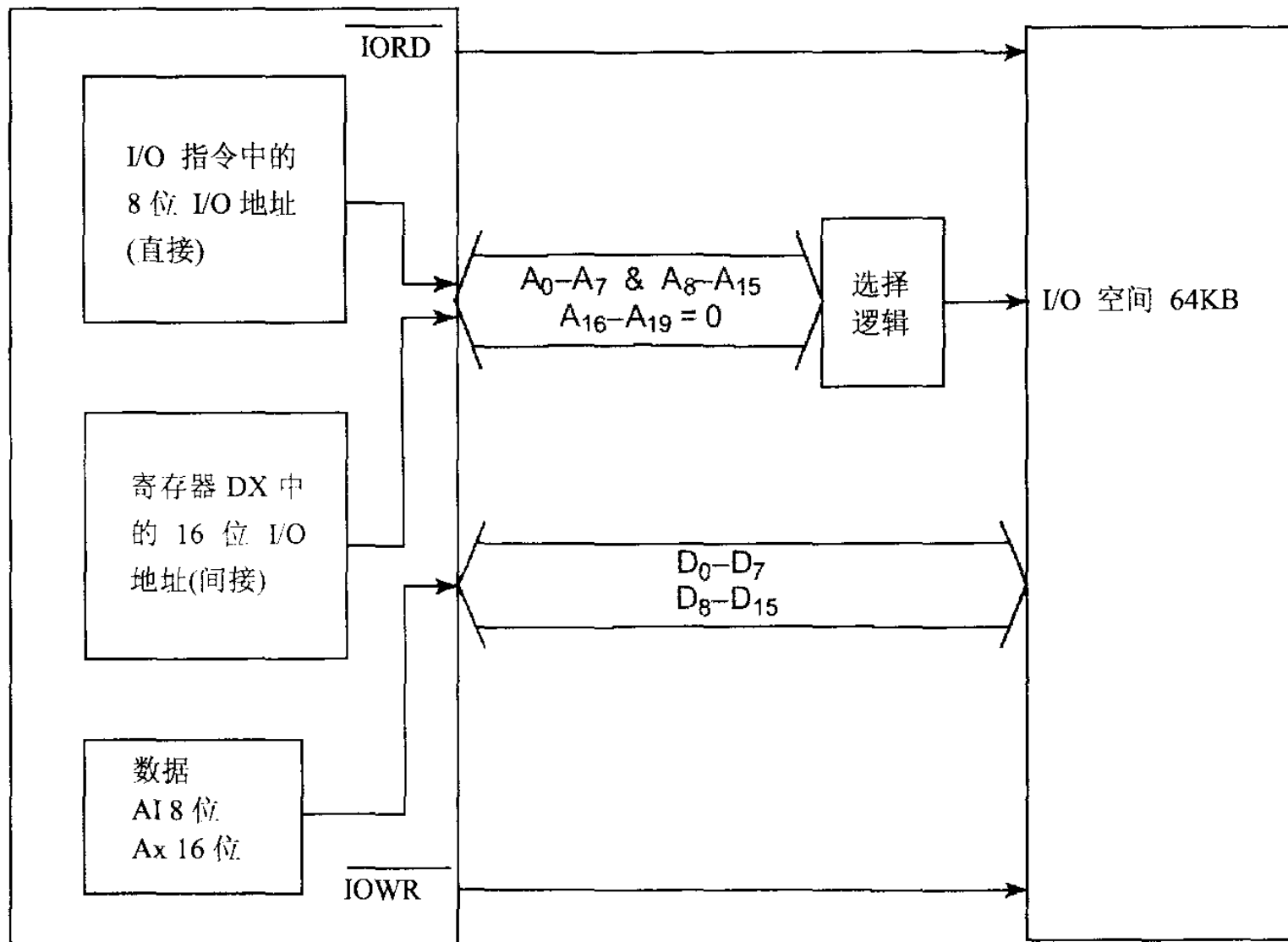
典型的总线连接



8086 CPU与I/O连接

- 8086微处理器能访问64KB或者32K字（两个字节）的I/O寄存器。I/O设备寻址能力的限制源于I/O寻址的地址宽度为16位，也就是说I/O设备的最大存储容量为64KB。I/O地址在T1状态时出现在地址总线的A0到A15上，然后被ALE信号锁存。在I/O操作时，地址总线的高端（A16~A19）总是逻辑0。
- 16位寄存器DX被用作16位 I/O地址指针，最大的寻址能力刚好达到64KB。存储映射I/O接口被当作存储单元被微处理器访问，数据的偶地址字节在D7~D0上传输，奇地址字节在D15~D8上传输。在设计8086系统的8位I/O子系统时需要注意系统中所有的字节寄存器的地址都应为偶数。

8086的I/O访问模式



8086 CPU的中断系统

- 8086 CPU有一个简单而灵活的中断系统，采用向量型中断结构，可以处理多达256个不同类型的中断请求。CPU的中断源有两类，**内部中断和外部中断**，**外部中断又分为非屏蔽中断NMI和可屏蔽中断INTR**。非屏蔽NMI中断请求信号为上升沿触发，一旦该信号有效，在现行指令执行完后立即中断，执行对应的中断处理程序，不需中断响应周期。而**可屏蔽中断通过INTR向CPU发出中断请求信号**，如果CPU开放了中断，在执行完当前指令之后就进入中断响应周期，中断响应周期占用**两个总线周期**。

内部中断（软件中断）

- 溢出中断（向量号为**4**的内部中断）
- 除法出错中断（向量号为**0**的内部中断）
- 单步中断（向量号为**1**的内部中断）

8086没有直接对TF置1或清0的命令，可修改存放在栈中标志内容，再通过POPF指令改变TF的值

- 断点中断（向量号为**3**的内部中断）
- **指令中断**（向量号为**n**的内部中断）

指令中断是执行INT n时，产生一个向量号为n的内部中断，为两字节指令，INT 3除外。INT n主要用于系统定义或用户自定义的软件中断，如BIOS功能调用和DOS功能调用。

内部中断向量号除指令中断由指令指定外，其余都是预定好的，因此都不需要传送中断向量号，也不需要中断响应周期。

外部中断（硬件中断）

- CPU外部中断请求信号引脚上输入有效的中断请求信号引起的，分为非屏蔽中断NMI和可屏蔽中断INTR

1) 非屏蔽中断NMI

- 非屏蔽中断NMI是由**NMI引脚上输入有效的中断请求信号**引起的一个**向量号为2的中断**。NMI用来通知CPU发生了致命性事件，如**电源掉电、存储器读写错、总线奇偶位错**等。NMI是不可用软件屏蔽的，而且是上升沿触发的，**中断类型号预定为2**，不需要中断响应周期。在IBM PC系列机中，NMI用于处理存储器奇偶校验错、I/O通道奇偶校验错以及8087协处理器异常中断等。

外部中断（硬件中断）

2) 可屏蔽中断INTR

- CPU的INTR引脚通常由**8259A** PIC的**INT输出**信号驱动，8259A又与需要请求中断的外设相连。在外设发出中断请求信号时8259A根据优先权和屏蔽状态，决定是否发出INT信号。外设的中断请求信号必须在中断请求信号被接受前保持有效。而CPU对INTR信号是在每条指令的最后一个时钟周期采样的。
- 接到有效的INTR信号后，CPU是否响应该中断请求，取决于中断允许标志位IF的状态。若IF=1，CPU开放中断，则响应，否则不响应。因此，要响应INTR的中断请求，CPU必须开放中断。
- 8086设有对中断标志位IF置1或清0的指令，**STI指令给IF置1，CPU开中断；CLI指令给IF清0，CPU关中断**。

硬件中断和软件中断的区别

- 若按中断源来划分，则系统中的中断又可分为：**硬件中断和软件中断**。而对于硬件中断，程序员不能控制它，它们基本上是随机产生的，而对于软件中断，程序员可通过指令INT有目的的安排它们的
- 两者的区别有以下几点：
 - 1、软中断发生的时间是由程序控制的，而硬中断发生的时间是随机的；
 - 2、软中断是由程序调用发生的，而硬中断是由外设引发的；
 - 3、硬件中断处理程序要确保它能快速地完成它的任务，这样程序执行时才不会等待较长时间。

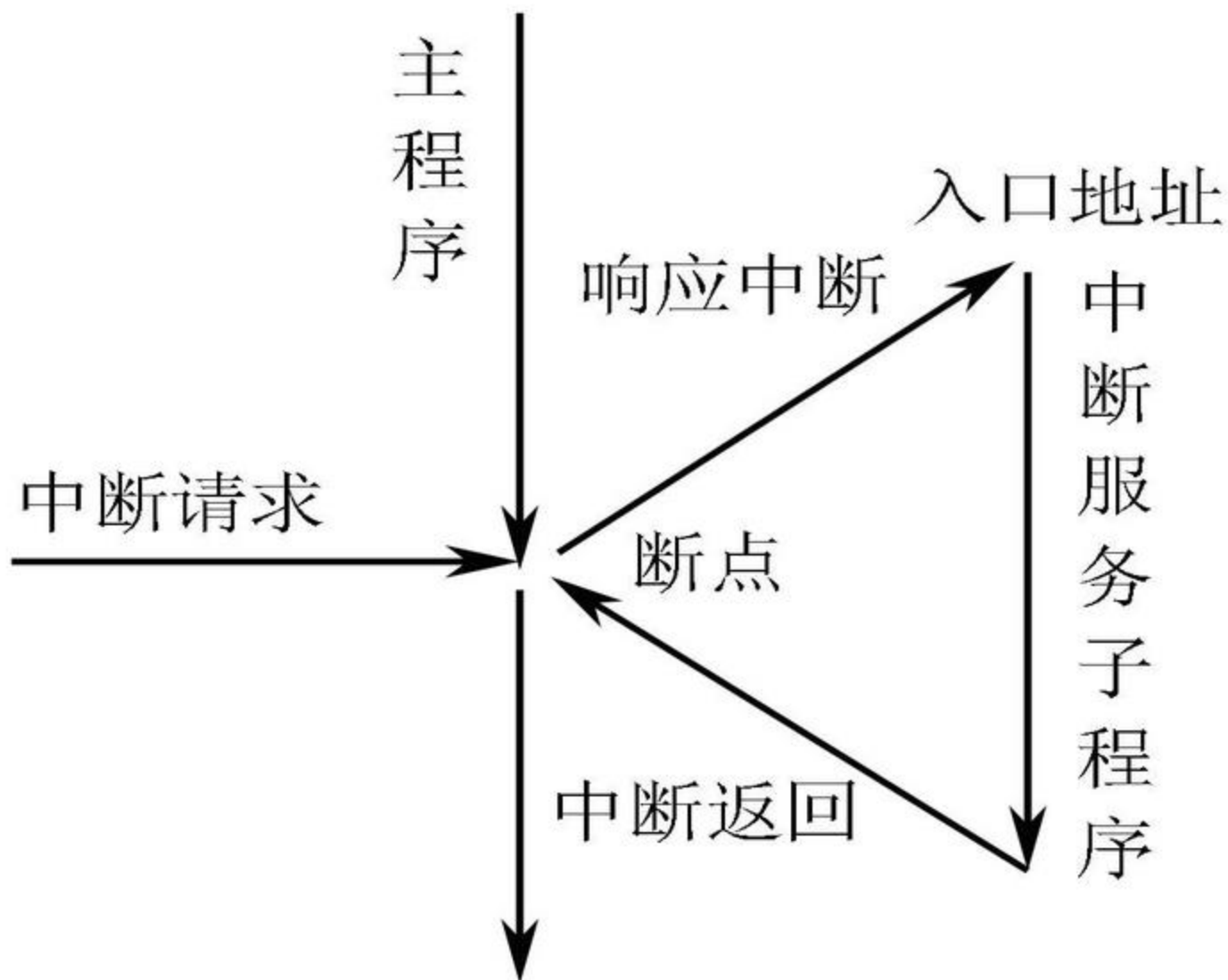
可屏蔽中断处理过程

- 可屏蔽中断处理的过程一般分为几步：**中断请求；中断响应；保护现场；转入执行中断服务子程序；恢复现场和中断返回。**

CPU响应中断要有几个条件：

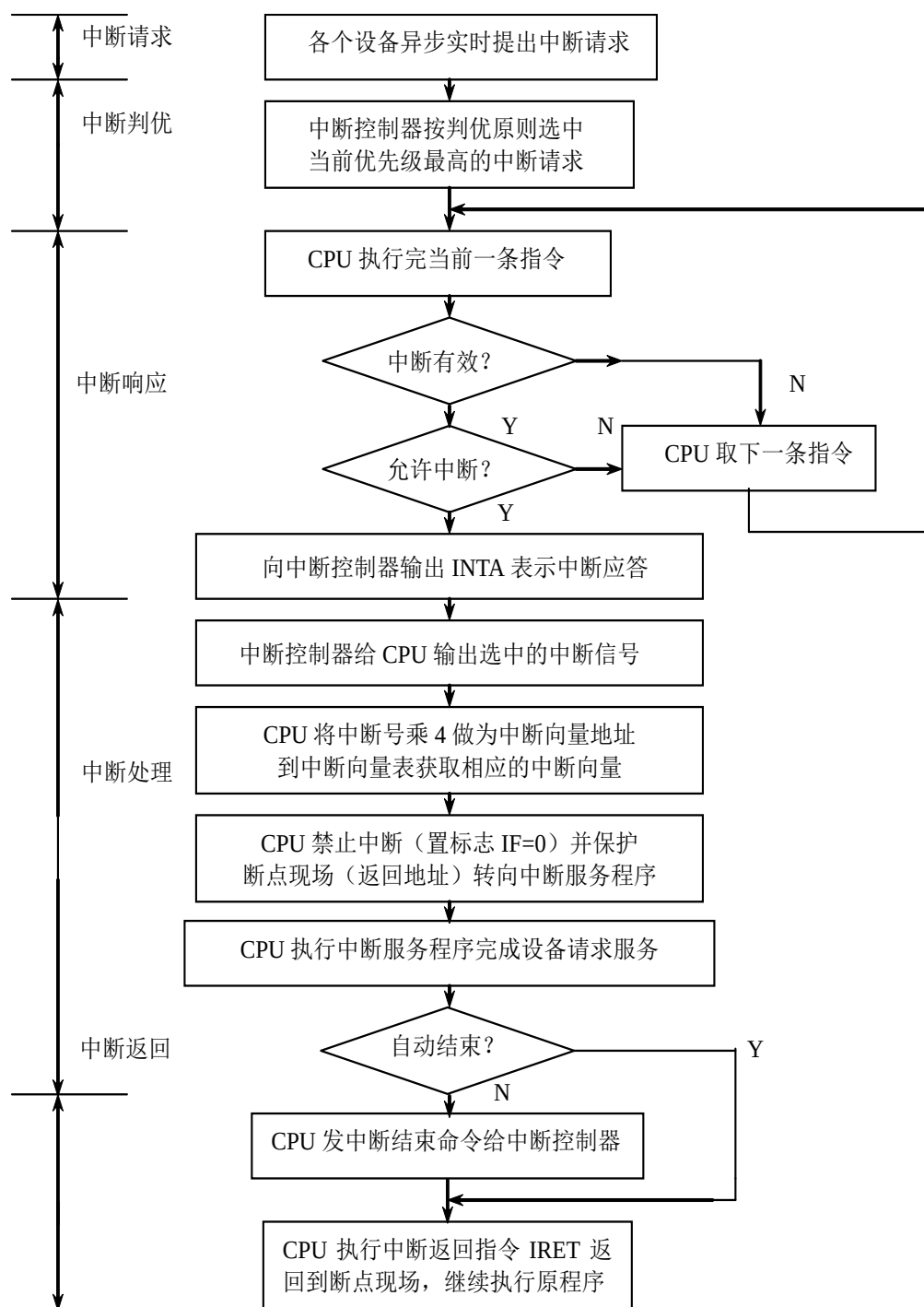
- 1) 外设提出中断申请；
- 2) 本中断位未被屏蔽；
- 3) 本中断优先级最高；
- 4) CPU允许中断。

8086 CPU的中断系统



响应中断流程

- 当中断接口电路中的**中断屏蔽触发器未被屏蔽**时，外设可通过中断接口发出中断申请。外设发出中断请求的时间是随机的，而CPU在每条指令的最后一个机器周期的最后一个T状态去采样中断请求输入线INTR，当CPU在**INTR引脚上接收到一个有效的中断请求信号**，而**CPU内部中断允许触发器是开放的**，则在当前指令完成后CPU响应中断。CPU响应中断后，对中断接口电路发出**两个中断响应信号** $\overline{INTA}$ 当中断接口电路收到**第二个** $\overline{INTA}$ 以后，通过数据线向CPU送中断类型号。



响应中断的四个阶段

1、中断请求与响应阶段

- CPU在每条指令执行的最后一个机器周期采样中断请求信号，在执行完当前指令后，进入是否响应中断的判断流程，如果是内部中断或NMI非屏蔽中断，**CPU自动形成中断类型号**，如果是INTR可屏蔽中断，进入中断响应周期，**从数据总线获取中断类型号**。

2、中断自动处理阶段

- 标志寄存器FR入栈，令TEMP=TF，暂存TF的状态，IF和TF清0，CS和IP入栈，根据中断类型号查中断向量表，都是硬件自动完成的。

3、中断服务阶段

- 主要是执行相应的中断服务程序，所做的处理视应用场合而定，如外设的中断服务程序，主要是传递信息，而软件启动的中断服务程序则主要为系统中其它程序服务。

4、中断返回

- 当执行到IRET指令时，自动弹出IP和CS以及标志寄存器FR，返回中断前的程序位置，执行下一条指令。

中断向量表

- 8086中断系统采用的是向量型中断方式，每个中断源都有一个为它服务的中断服务程序。8086最多能管理**256个中断**，对应类型为**00H~FFH**，称**中断类型号或中断向量号**，是识别中断源的唯一标志。
- 每一个中断服务程序都有一个确定的入口地址，该地址称为**中断向量**。
- 把系统中所有的**中断类型号**及其对应的**中断向量**按一定的规律存放在一个区域内（1KB）**00000H~003FFH**，这个区域叫**中断向量表**，亦即**中断入口地址表**。

中断向量表

专用中断	000	类型0中断入口(除法出错)	IP
	004	类型1中断入口(单步中断)	CS
	008	类型2中断入口(NMI)	IP
	00C	类型3中断入口(断点中断)	CS
	010	类型4中断入口(溢出中断)	
	014	类型5中断入口	
系统备用	⋮	⋮	
	07F		
供用户使用	080	类型31中断入口	
		类型32中断入口	
	⋮	⋮	
	3FC		
		类型255中断入口	IP CS
	15	8 7	0

中断向量表

- 8086系统中，在存储器的00000H~003FFH地址范围建立了一个中断向量表，占用1K字节空间，**每个中断向量占用4个存储单元，4个单元中的前2个单元存放的是中断服务程序所在段内的偏移量（IP的内容，16位地址），低位字节存放在低地址，高位字节存放在高地址；后2个单元存放的是中断服务程序所在段的段基地址（CS的内容，16位地址），存放方法与前2个单元相同。CPU响应中断时，从中断向量表中查出中断向量地址，再从该地址中取出内容分别装入IP和CS，从而转去执行相应的中断服务程序。（专用中断类型号0~4 共5个、系统备用中断类型号5~31共27个、供用户使用224个）**

系统备用中断类型号

- 专为IBM PC机开发的基本输入输出系统BIOS中断调用占用10H~1AH共11个中断类型号，如**INT 10H**为屏幕显示调用，**INT 13H**为磁盘I/O调用，**INT 16H**为键盘输入调用，**INT 1AH**为时钟调用等，就是双字节指令中断，这些中断为用户提供直接与I/O设备交换信息的一系列子程序。DOS中断占用20H~3FH共32个中断类型号（其中A0~BBH和30H~3FH为DOS保留类型号），如DOS系统功能调用（**INT 21H**）主要用于对磁盘文件的存储管理。

中断向量表题目

- 8086中断类型号为40H的中断服务程序入口地址存放在中断向量表的起始地址是：

A、DS: 0040H

B、ES: 0100H

C、0000: 0100H

D、0000: 0160H

请列出详细解答步骤。

- 8086系统在存储器的00000H~003FFH建立一个中断向量表
- MOV AX, 0
- (XOR AX, AX)
- MOV DS, AX
- (MOV ES, AX)

中断类型号n设置中断向量

利用指令来实现

- MOV AX, 0
- MOV ES, AX
- MOV BX, N*4
- MOV AX, OFFSET RKDZ ; 取中断服务程序的偏移地址
- MOV ES: WORD PTR [BX], AX
- MOV AX, SEG RKDZ ; 取中断服务程序的段基地址
- MOV ES: WORD PTR [BX+2], AX

RKDZ:

:

IRET

中断类型号n设置中断向量

利用DOS功能调用来实现

- **设置中断向量：**把由AL中指定中断类型号的中断向量DS: DX，放置在中断向量表中。

预置：AL=中断类型号

DS: DX = 中断服务程序入口地址

AH = 25H

执行：INT 21H

- **取中断向量：**

预置：AL=中断类型号

AH = 35H

执行：INT 21H

返回：ES: BX = 中断服务程序入口地址

利用DOS功能调用设置中断向量和取中断向量

MOV AL, N ;先把原来中断向量表中的值取出来放到ES: BX中

MOV AH, 35H

INT 21H

设置中断向量:

PUSH ES ;存原中断向量

PUSH BX

MOV AX, SEG RKDZ ;设置中断向量段地址在DS

MOV DS, AX ;RKDZ 为中断服务子程序

MOV DX, OFFSET RKDZ ;设置中断向量偏移地址在DX

MOV AL, N ;中断类型号n

MOV AH, 25H ;设置中断向量

INT 21H

.....

;用户编写的主要程序，这里面会调用上面的中断子程序

.....

等程序快结束了，记得还要恢复原中断向量:

POP DX ;恢复原中断向量

POP DS

MOV AL, N

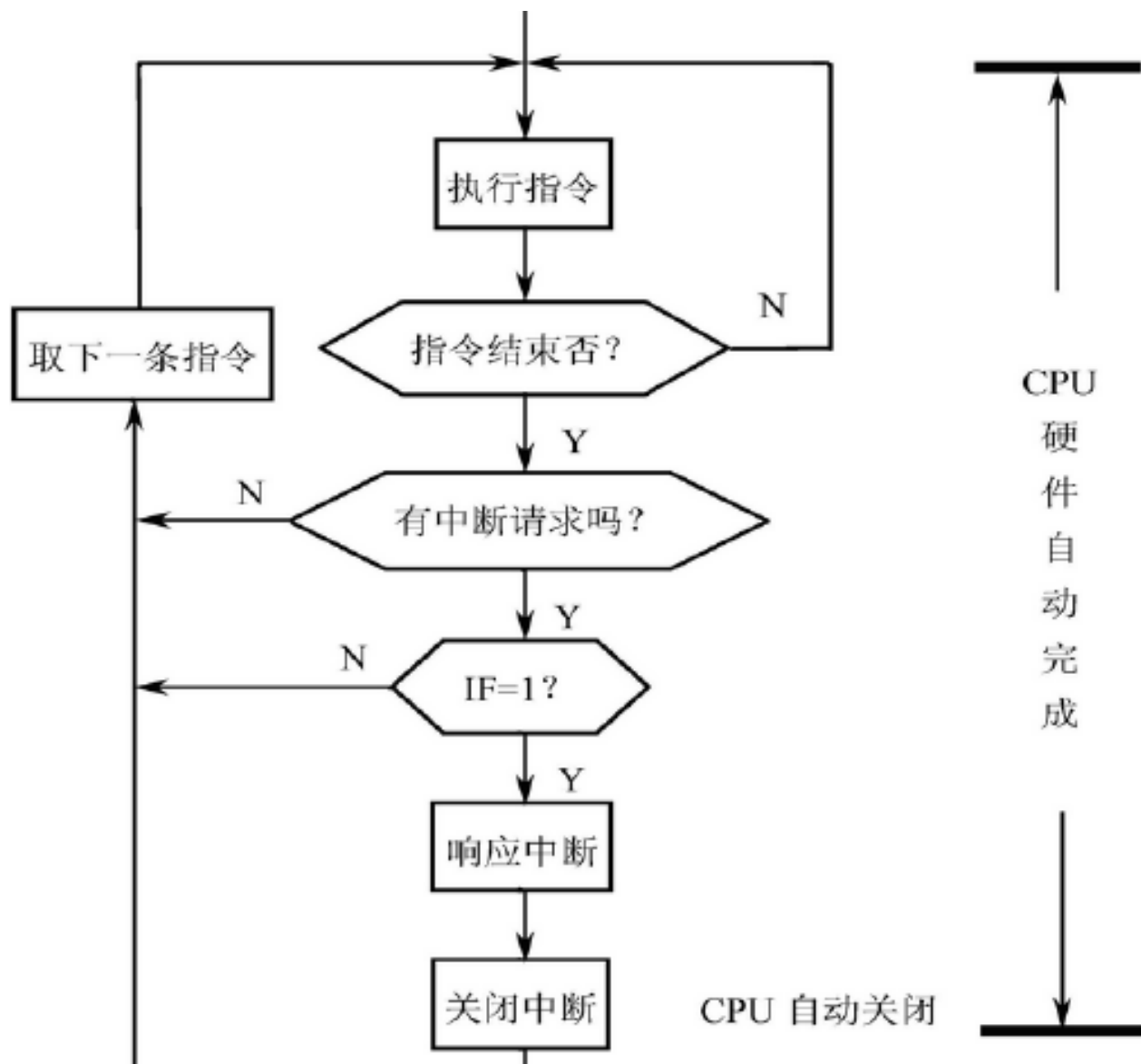
MOV AH, 25H

INT 21H

中断类型号的获取

- 矢量中断中，中断入口地址与中断类型号有关，那么中断类型号如何获取呢？
 - （1）对于除法出错，单步中断，不可屏蔽中断NMI，断点中断和溢出中断，CPU分别自动提供中断类型号0~4。
 - （2）对于用户自己确定的软件中断INT n，类型号由n决定。
 - （3）对外部可屏蔽中断INTR，可用硬件电路（例如，通用并行接口芯片8212）设计产生中断类型号
 - （4）对外部可屏蔽中断INTR，可用可编程中断控制器8259A获得中断类型号。

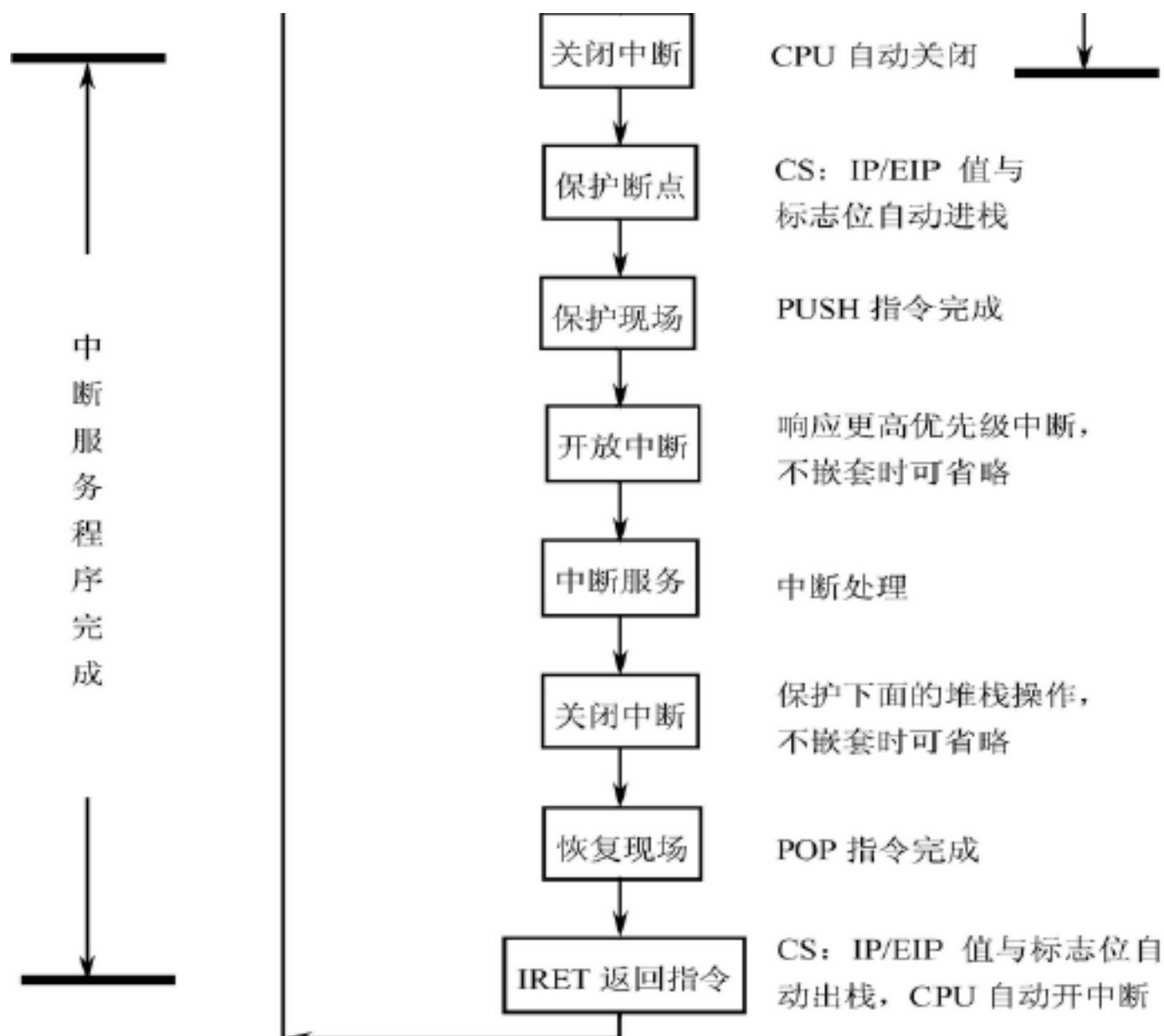
中断类型号的获取



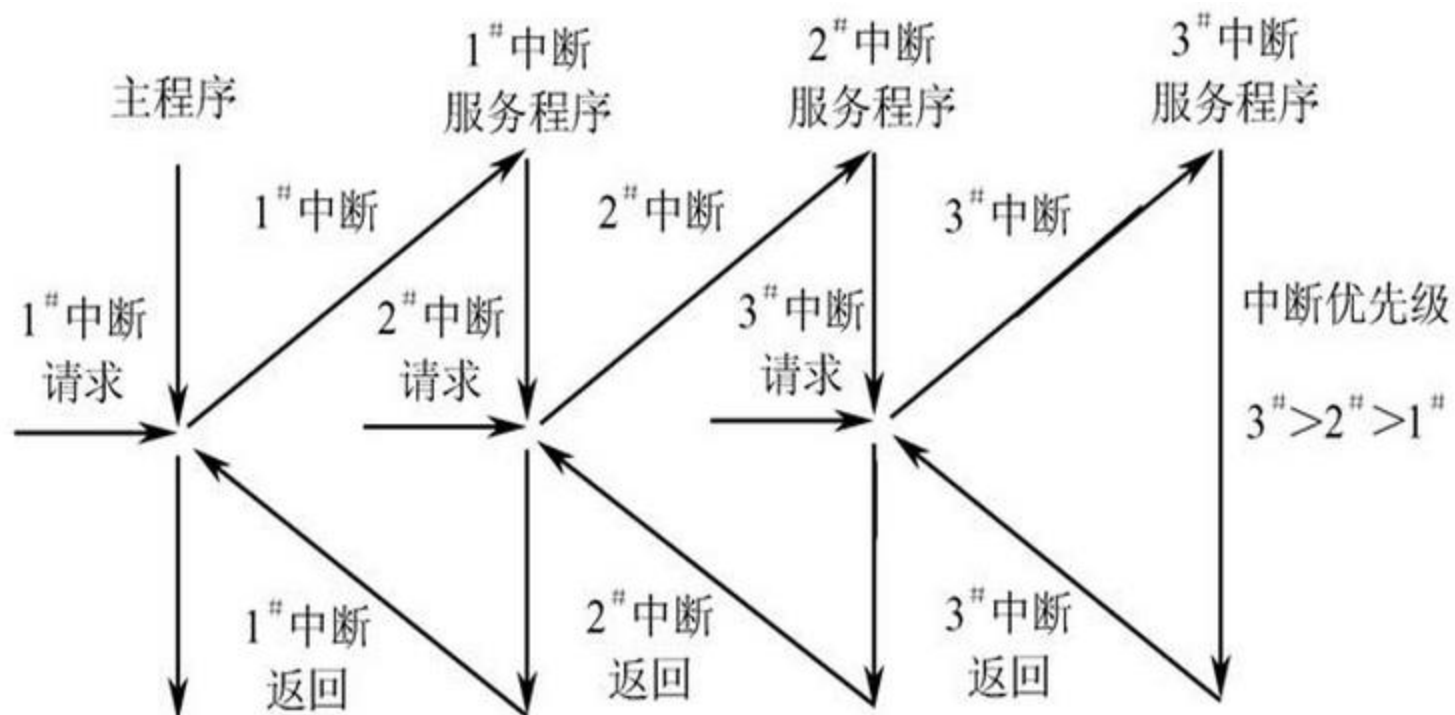
中断服务子程序

- 程序开始必须保护中断现场，可以通过一系列PUSH指令将CPU各寄存器的值入栈保护；
- 若允许中断嵌套，则用**STI指令来设置开中断**，使中断允许标志IF=1；
- 执行中断处理程序；
- 用**CLI指令来设置关中断**，使中断允许标志IF=0，禁止其他中断请求进入；
- 给中断命令寄存器送中断结束命令EOI，使当前正在处理的中断请求标志位被清除，否则同级中断或低级中断的请求仍会被屏蔽掉；
- 恢复中断时的现场，通过一系列POP指令将CPU各寄存器的值恢复；
- 用中断返回指令IRET返回主程序，此时堆栈中保存的断点值和标志值分别装入IP、CS和flags。

中断服务子程序



中断嵌套过程



低优先级中断源向CPU发出中断请求，且得到CPU的响应，若此时是低优先级或同级中断源发出的中断请求，CPU均不响应。这种高优先级中断源中断低优先级中断源的服务，使中断服务子程序可以嵌套进行的过程，称为**中断嵌套**。

中断服务子程序和普通子程序区别

它们的主要差异有：

- 1、中断服务程序和正在执行的程序之间是**相互独立的**；普通子程序与CPU正在执行的主程序之间是**同一程序的两个部分**。
- 2、子程序调用一定是程序员在编写程序时事先安排好的，是**可预见**的，而中断是由中断源根据自身的需要产生的，是**不可预见**的。除了软中断(如指令INT引起的中断)，通常中断都是随机产生的，而子程序调用是通过**调用指令**引发的。
- 3、子程序调用是用**CALL指令**来实现的，调用的入口地址通过程序调用指令指出（即子程序的标号），但中断服务程序的调用，首先是向CPU发出**中断请求事件**，由一系列的条件决定是否相应中断(INT指令也要理解为发出内部中断信号)，中断服务程序的入口地址也是由**中断向量号查表**指出。

中断服务子程序和普通子程序区别

- 4、中断服务程序是随机发生的，强调中断（有临时终止断开、打断的意思），结束后要返回断点。普通子程序是安排好的没有断点。（排队和插队）
- 5、在主程序中，赋**入口参数之前**压栈保护。同理，也应在主程序中，处理完出口参数后，出栈恢复现场。而中断子程序的保护现场和恢复现场**只能在中断子程序中完成**。且在保护和恢复现场时，不能允许高级中断的产生，在恢复现场前须关中断，即关中断和关单步中断。
- 6、处理中断服务程序时，对多个同时发生的中断要进行裁决，而调用子程序时一定没有这种操作。
- 7、子程序的返回指令是RET，而中断服务程序的返回指令是IRET/RETI。
- 8、在通常情况下，子程序是由应用程序开发者编写的，而中断服务程序是由系统软件设计者编写的。

中断服务子程序

INT-P	PROC		; 中断服务子程序
PUSH	DS		; 保护现场
PUSH	AX		
PUSH	DX		
STI			; 开中断
MOV	DS, AX		
MOV	DX, OFFSET MESS		
MOV	AH, 09H		
INT	21H		
DEC	COUNT		; 控制10次循环
JNZ	NEXT		
IN	AL, 21H		; 读IMR
OR	AL, 04H		; 屏蔽IR2请求
OUT	21H, AL		
NEXT:	CLI		; 关中断
MOV	AL, 20H		; 写OCW2, 送中断结束命令EOI
OUT	20H, AL		
POP	DX		; 恢复现场
POP	AX		
POP	DS		
IRET			; 中断返回
INT-P	ENDP		

简单的中断子程序实验

一. 实验内容

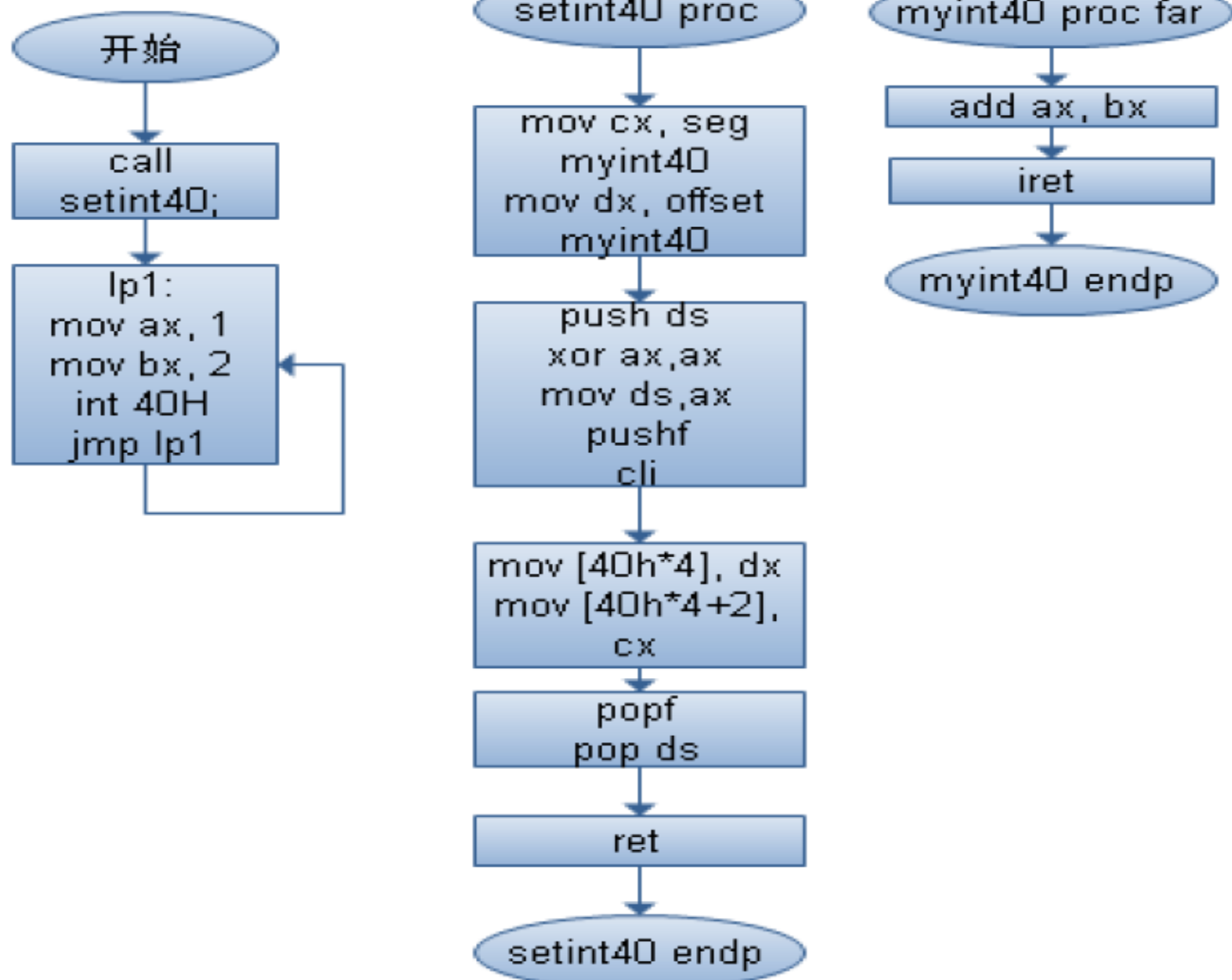
- 实现子程序和中断服务子程序的设计
- 编写子程序设置**40H**中断为自定义中断服务子程序
- 编写中断服务子程序实现 **$AX=AX+BX$**
- 编写主程序调用子程序和中断

二. 扩展要求

- 保留原中断服务子程序功能

三. 思考内容

- 中断向量表结构
- 中断服务程序链的实现



```
CODE    SEGMENT
        ASSUME  CS:CODE, DS:CODE, SS:CODE
```

```
START:  JMP  GO
```

```
        DW  100  DUP(?)
```

```
TOS     EQU  THIS  WORD
```

```
OLDIP   DW  ?    ;用于存放原 INT9 中断向量（即中断服务程序入口地址）中的偏移值
```

```
OLDCS   DW  ?    ;用于存放原 INT9 中断向量（即中断服务程序入口地址）中的段值
```

```
GO:
```

```
        MOV     AX, CS
```

```
MOV     DS, AX
```

```
MOV     SS, AX
```

```
MOV     SP, OFFSET TOS
```

```
MOV     AL, 9H      ;取原中断向量
```

```
MOV     AH, 35H
```

```
INT     21H
```

```
MOV     OLDCS, ES    ;存原中断向量段址
```

```
MOV     OLDIP, BX    ;存原中断向量偏移量
```

PUSH DS

MOV DX, OFFSET MYINT9 ;新 INT9 中断向量的偏移量送 DX

MOV AX, SEG MYINT9

MOV DS, AX ;新 INT9 中断向量的段值送 DS

MOV AL, 9H

MOV AH, 25H

INT 21H ;新 INT9 的中断向量写入中断向量表

POP DS

IN AL, 21H

AND AL, 11111101B

OUT 21H, AL ;确保中断屏蔽寄存器没有屏蔽键盘中断

STI

NEXT: JMP NEXT ;主程序死循环

MOV AH, 4CH

MYINT9 PROC FAR

PUSH AX

PUSH DX

STI

IN AL, 60H

TEST AL, 80H

JNZ EXIT

这里只考虑按下键。

;如果接受的扫描码最高位为 0，表示按下键，否则是释放键，

MOV BL, AL

MOV DL, AL

SHR DL, 1

SHR DL, 1

SHR DL, 1

SHR DL, 1

CMP DL, 9

JBE L

ADD DL, 7

L: ADD DL, 30H

MOV AH, 2

INT 21H

;输出扫描码字节的高 4 位（1 位十六进制数）

MOV DL, BL

AND DL, 0FH

CMP DL, 9

www.docin.com

```
MOV    AH, 2
MOV    DL, 0AH
INT     21H
MOV    DL, 0DH
INT     21H      ;显示回车换行
```

EXIT:

```
MOV AL, 20H
OUT 20H, AL
```

```
POP DX
POP AX
```

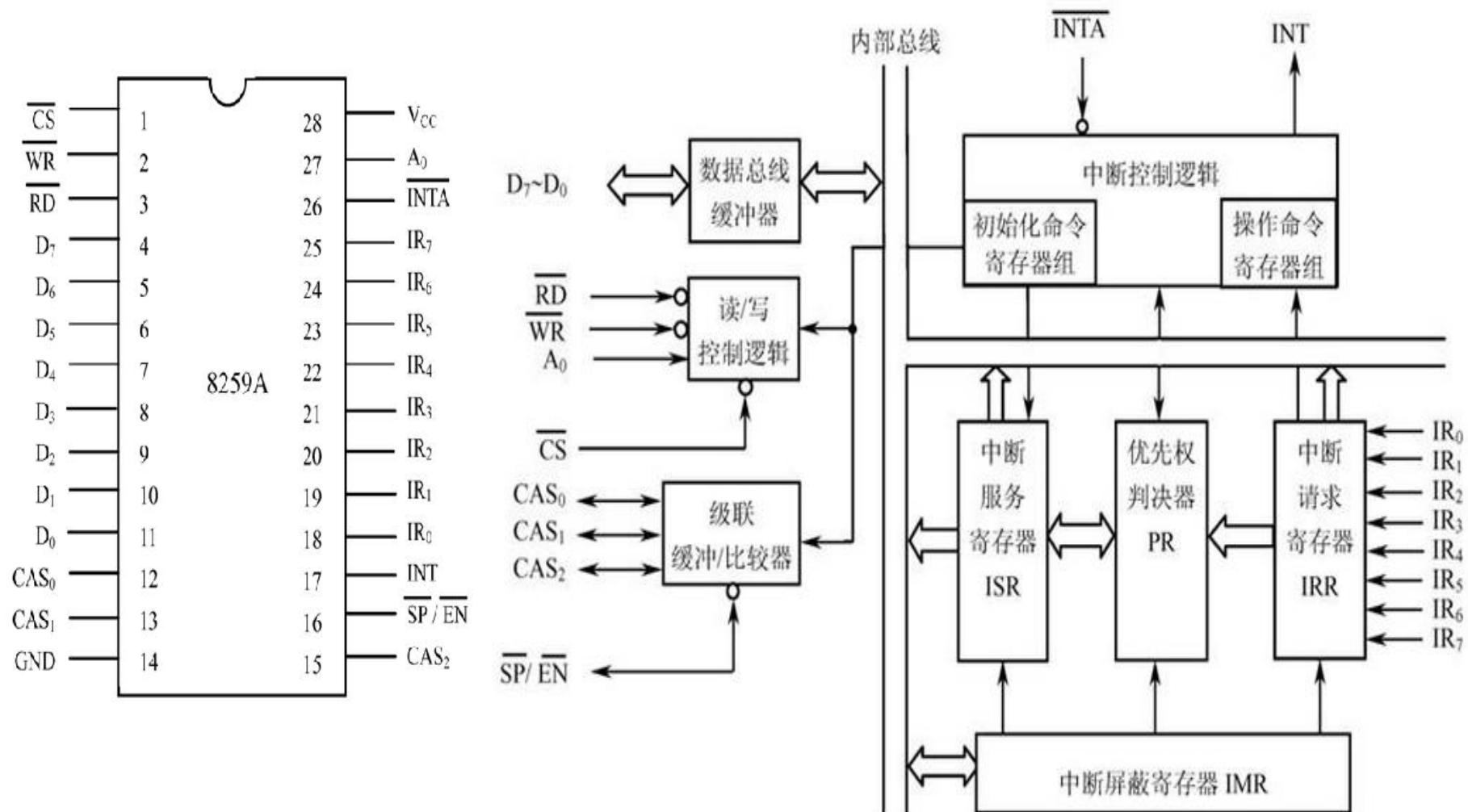
```
IRET
```

MYINT9 ENDP

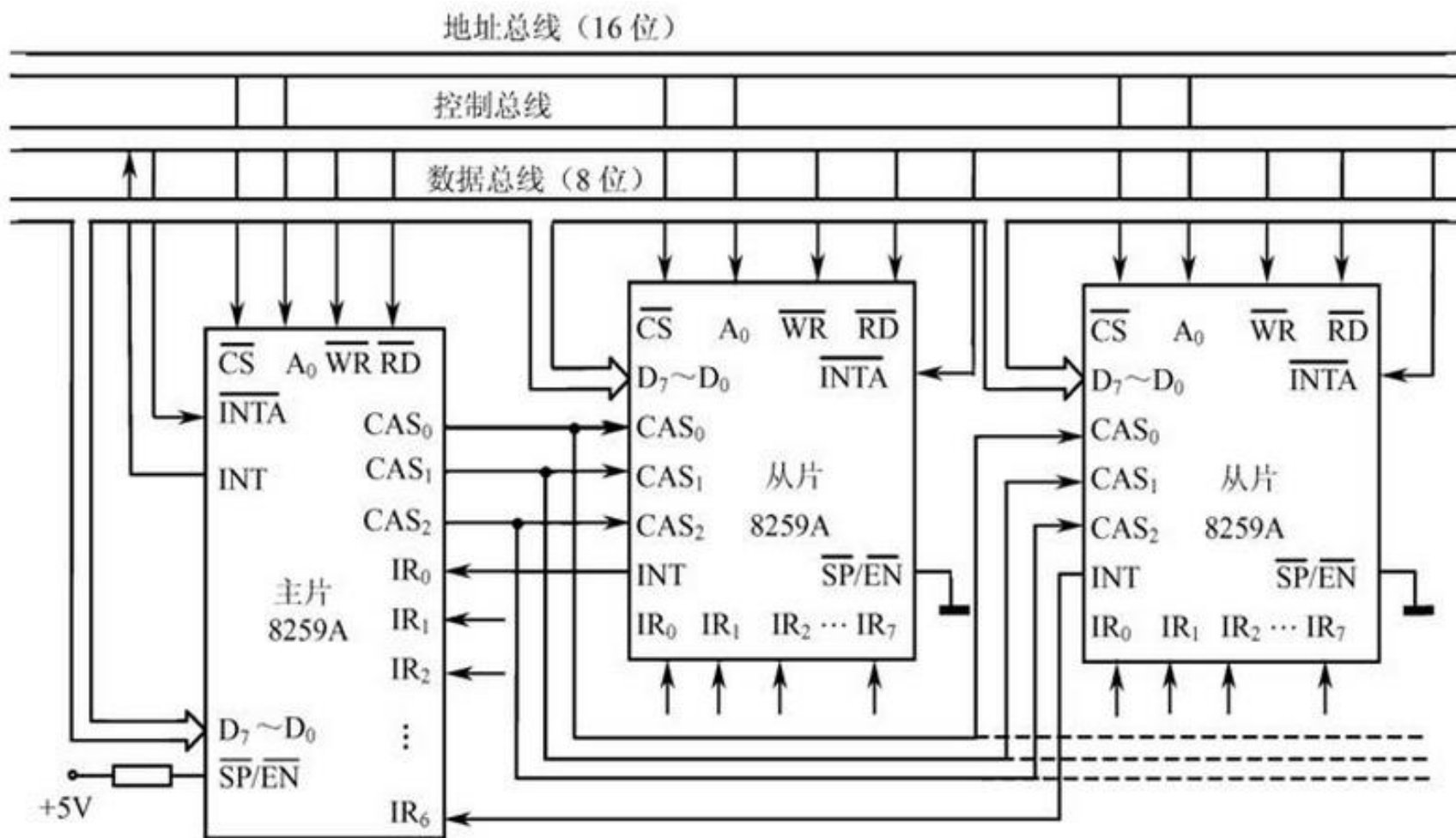
可编程中断控制器8259A

- 可编程中断控制器8259A是Intel公司专为80x86 CPU控制**外部中断而设计开发的芯片**。
- 它将中断源优先级判优、中断源识别和中断屏蔽电路集于一体，不需要附加任何电路就可以对外部中断进行管理，单片可以管理8级外部中断，在多片级联方式下，可以管理多达64级的外部中断。

8259A内部结构及引脚信号



8259A多片级联方式硬件连接



学习网站地址

<http://www.tyut.edu.cn/kecheng1/2008/site04/courseware/chapter5/5-4-4.html>



太原理工大学
TAIYUAN UNIVERSITY OF TECHNOLOGY

微机原理与接口技术多媒体教学课件

首 页

第一章

第二章

第三章

第四章

第五章

第六章

第七章

第八章

第九章

附 录

位置: 目录-第五章-第四节

5.4.4 存储器的扩展设计举例

- 第一节 存储器的分类与性能指标
- 第二节 只读存储器
- 第三节 随机存取存储器
- 第四节 存储器的扩展设计
- 第五节 高速缓冲存储器
- 第六节 虚拟存储器及其管理技术
- 第七节 存储器管理模式

微机原理与接口技术多媒体教学课件

首 页

第一章

第二章

第三章

第四章

第五章

第六章

第七章

第八章

第九章

附 录

位置: 目录-第五章-第四节

- 第一节 中段概述
- 第二节 80486微处理器中断系统
- 第三节 可编程中断控制器8259A